NFL Player Performance: design document

A data pipeline, prediction engine and website that rank NFL players by position and project each week's results. Live at nflstats.johnwasikye.com. Source code: github.com/John-Wasikye/nfl-player-performance.

Template: Google design doc, arc42 sections, C4 model, the same format as the platform design doc for *Will I have signal out there?* (17 sections, requirements with IDs, a decision table, runbooks, glossary). **Citations:** bracketed keys name the source of a fact, listed in the table below. Requirement IDs (FR-nn, QR-nn), project rules (R-nn), decisions (D-nn), risks and open questions (OQ-n) are assigned in this document.

Names of files, buckets, commands and tables in this document are the ones in the repository and the deployed system at the time of writing. Where a source is older than another, the newer source wins and appendix G records the difference.

Source key

Key	Source
[README]	README.md on the main branch of the repository (raw file)
[CLAUDE]	CLAUDE.md, the project's standing instructions (commands, conventions, storage rules)
[OPS]	docs/operations.md, written 2026-10-06: schedule, alerts, runbook, known gaps
[STG]	docs/storage-design.md, decided 2026-10-01, built 2026-10-05: state classes, buckets, write-once records
[NSF]	docs/no-silent-failures.md: the standing rule and the 19-item failure inventory
[RES]	docs/prediction-research.md: the research behind the prediction design (sections 1 to 10)
[BT]	docs/backtest.md, generated 2026-10-06T05:24 UTC
[INF]	infra/README.md: Terraform, CI/CD, IAM roles, DNS
[WEB]	web/README.md: the website
[DEP]	docs/dependencies.md: held-back dependencies
[CON]	pipeline/nfl_pipeline/contract.py: the published data contract, schema version 1
[SEED]	dbt/seeds/ranking_config.csv and ranking_weights.csv: the ranking method
[CI]	.github/workflows/ci.yml; [DOCK] Dockerfile; [PYP] pyproject.toml
[SITE]	johnwasikye.com project pages and nflstats.johnwasikye.com (home, methodology, report card)

> **Not available when this was written:** BUILD_PLAN.md and SESSION_CONTEXT.md (kept outside the repository on John's computer), the Terraform files, the pipeline and dbt source, the tests, and the workflow files plan.yml and deploy.yml. The repository's directory pages were blocked to automated reading, so this document is built from the documents above, which are detailed. Statements about those files are second-hand and cite the document that describes them.

Contents

1 Status, owner, reviewers, last updated . . . 4

2 Summary . . . 4

3 Context, scope and constraints . . . 5

3.1 Background facts . . . 5

3.2 System boundary . . . 5

3.3 Constraints . . . 6

4 Goals and non-goals . . . 6

5 Requirements . . . 6

5.1 Functional requirements . . . 7

5.2 Quality requirements . . . 8

5.3 Standing project rules . . . 9

5.4 Phases, status and acceptance evidence . . . 9

5.5 Traceability . . . 10

6 Architecture . . . 11

6.1 System context (C4 level 1) . . . 11

6.2 Containers (C4 level 2) . . . 12

6.3 Components of the daily task (C4 level 3) . . . 13

6.4 Repository structure . . . 13

7 Component breakdown . . . 13

7.1 Ingest: pipeline/nfl_pipeline/ . . . 13

7.2 Transform: dbt/ on DuckDB . . . 14

7.3 Validation gate and publish . . . 14

7.4 Backtest . . . 15

7.5 Prediction engine: pipeline/nfl_pipeline/predict/ . . . 15

7.6 Records and locking . . . 17

7.7 Experiment loop . . . 18

7.8 Website: web/ . . . 19

7.9 Infrastructure and storage: infra/ . . . 19

7.10 CI/CD: .github/workflows/ . . . 20

7.11 Monitoring and alarms . . . 20

8 Runtime view . . . 22

8.1 Daily run (15:00 UTC) . . . 22

8.2 Public request . . . 23

8.3 Deploy flow . . . 24

8.4 Weekly cycle . . . 24

9 Data model . . . 24

9.1 Published files . . . 24
9.2 Contract models (selected fields) . . . 25
9.3 Warehouse . . . 25
10 API . . . 25
11 Deployment view . . . 26
12 Cross-cutting concerns . . . 26
12.1 Security . . . 26
12.2 Privacy . . . 27
12.3 Observability . . . 27
12.4 Licensing and attribution . . . 27
12.5 Cost . . . 27
12.6 Reliability . . . 27
12.7 Time zones . . . 28
12.8 Accessibility . . . 28
12.9 Dependencies . . . 28
13 Testing strategy . . . 29
14 Alternatives considered . . . 30
15 Risks . . . 31
16 Open questions and known gaps . . . 31
17 Glossary . . . 33
Appendix A: Ranking metrics by position . . . 34
Appendix B: Backtest tables . . . 34
Appendix C: Prediction research summary . . . 35
Appendix D: Operations runbook . . . 36
Appendix E: Command reference . . . 38
Appendix F: Held-back dependencies . . . 38
Appendix G: Source discrepancies and coverage gaps . . . 39
Appendix H: Sources . . . 39

1 Status, owner, reviewers, last updated

Field	Value
Status	Draft, awaiting John's review. Describes the system as built and deployed.
Scope	NFL Player Performance, phases 1A to 1D (local pipeline, website, weekly predictions, AWS deployment). Phase 1E (launch polish) is named in [CLAUDE] with no status given. Phase 2, a mobile app, is planned and not started.
Owner	John Wasikye (GitHub: John-Wasikye)
Reviewers	John
Last updated	Oct 14, 2026. The newest repository document read is dated Oct 6, 2026, so anything that changed after that date (for example, whether a week has been locked and graded) is not reflected.
Build position	Written after the build. The project started in September 2026 [SITE]. The storage design was decided 2026-10-01 and has run on AWS since 2026-10-05 [STG]. The operations runbook is dated 2026-10-06 [OPS].

2 Summary

NFL Player Performance pulls NFL statistics from the open nflverse datasets every day, cleans and models them with dbt on DuckDB, and publishes position rankings (QB, RB, WR, TE, K) to a website. Each position is ranked two ways: a composite score (20% efficiency, 80% production, each metric percentile-scored within the position) and season-to-date PPR fantasy points. Every week is a snapshot, so rank movement is visible [README, SEED].

On top of the rankings sits a weekly prediction engine. It projects each eligible player's next game with an 80% range, locks the forecast before the first kickoff so it cannot be changed, and grades it afterwards in a public report card that compares it with simple baselines, including the weeks where it loses to a plain average. A governed experiment loop lets Claude Code propose one new feature at a time; an automated test on held-out weeks decides whether it ships. No published number is produced or adjusted by an LLM [README, RES].

The system runs by itself on AWS: EventBridge starts one Fargate task a day at 15:00 UTC, the task runs **nfl-pipeline daily** in about 90 seconds, and the results go to S3 behind CloudFront. Terraform builds the infrastructure and GitHub Actions deploys through OIDC with no stored AWS key. It was priced at about a dollar a month [README, OPS].

The project's rules, more than its code, are what keep its numbers credible: derived state is disposable and rebuilt every run, the two records (locked forecasts and the experiment ledger) are write-once, and every failure must announce itself, with silence treated as a failure [STG, NSF].

3 Context, scope and constraints

3.1 Background facts

Fact	Source
Live at nflstats.johnwasikye.com. Public repository John-Wasikye/nfl-player-performance. Started September 2026.	SITE, README
Data: 16 nflverse datasets as Parquet (schedules, players, weekly stats, play-by-play, injuries, snap counts, rosters, depth charts, participation, advanced stats, charting data, Next Gen Stats). Licence CC BY 4.0. No NFL.com data, no photos or logos.	README
Daily run at 15:00 UTC (11:00 AM Eastern during daylight time, 10:00 AM after the clocks change on 2026-11-01). About 90 seconds, a few cents per run.	OPS
Priced at about a dollar a month. A $5 budget and billing alarm exist, made by hand and not managed in Terraform.	README, INF
Region us-east-1. CLI profile nfl_player_stats (IAM Identity Center, about 12-hour sessions).	INF, OPS
Four S3 buckets with the prefix jw-nfl-player-stats-: raw, records, site, tfstate. All with Block Public Access.	STG, OPS
Terraform for all infrastructure. Applied by hand from the laptop. CI only plans.	INF, CLAUDE
GitHub Actions reaches AWS through OIDC. The only repository secret is AWS_ACCOUNT_ID, which grants nothing.	INF
Python 3.10+ locally, 3.12 in Docker. The same image runs locally and on AWS.	CLAUDE, DOCK
Tests: 268 Python tests, 158 dbt data tests, 59 web unit tests, 91 end-to-end tests including accessibility.	README
State sizes (2026-10-01): raw snapshots 140 MB, warehouse 163 MB, published JSON 1.6 MB, locked predictions about 60 KB a week, ledger 3.3 KB.	STG
Rankings use current-season data only. Past seasons feed the backtest and the predictions. Research used regular seasons 2021 to 2025; the 2026 season is in progress.	CLAUDE, RES
Published data contract is schema version 1. A change that breaks readers bumps SCHEMA_VERSION.	CON

3.2 System boundary

Inside the system: the Python pipeline (ingest, storage, publish, contract, predictions, CLI), the dbt project (staging, marts, ranking and feature models, seeds, data tests), the prediction engine and promotion gate, the published JSON contract, the Next.js static website, the Docker image, the Terraform in infra/, the GitHub Actions workflows, the alarms and runbook, and the research in research/ and docs/.

Outside: the nflverse datasets and their publisher; the NFL and its teams (the project is independent); AWS, GitHub, Cloudflare and Anthropic as providers; the mobile app (phase 2); and the weekly Claude analyst session, which runs on John's laptop inside the Claude Pro plan and reaches the system only as reviewed code [STG].

3.3 Constraints

- About a dollar a month on AWS. The pipeline makes no LLM API calls, and ANTHROPIC_API_KEY is never set, because that would silently switch Claude Code to paid API billing [RES, STG].
- Only nflverse's published release files. Never NFL.com. No player headshots or logos; avatars are initials on team colors [CLAUDE].
- A Fargate task starts with an empty disk and loses it on exit, so every file must either be rebuilt from raw or live in S3 [STG].
- No stored AWS keys anywhere: CI uses OIDC and the task uses its task role [INF, STG].
- A one-person project. Object Lock uses governance mode, not compliance mode, because a bad write is a realistic Tuesday [STG].
- The site is a static export. The browser fetches published JSON from /data/v1, so there is no application server [WEB].
- Python code must stay compatible with 3.10 [CLAUDE].
- Public text (site copy, docs, README, ledger wording) is written in first person singular, in plain words, for a reader who is a data engineer or a recruiter [CLAUDE].

4 Goals and non-goals

Goals

- **G1.** Rank QB, RB, WR, TE and K from raw public data every day, using only games through the ranked week [README].
- **G2.** Project each eligible player's next game with a range, lock it before kickoff, and grade it publicly against simple baselines [README, RES].
- **G3.** Nothing fails silently. Every failure announces itself, and the absence of a signal is itself an alarm [NSF].
- **G4.** Locked forecasts and the experiment ledger cannot be changed after the fact [STG].
- **G5.** Run unattended on AWS for about a dollar a month, with the laptop needed only for the weekly analyst session [README, STG].
- **G6.** Use AI only where the evidence supports it: feature proposals that must pass an automated gate, never published numbers [RES].
- **G7.** Test every change, and run the same Docker image locally and on AWS [README, CI].

Non-goals

- No betting or financial advice. The site says it provides statistical rankings only [SITE].
- No component predictions (targets, carries, yards, touchdown probabilities) yet. Only PPR points are published [RES section 7.1].
- No stadium-specific or team-specific home effects. Home is one league-wide feature [RES section 7.4].
- No self-tuning of predictions from recent errors. Every such mechanism tested made predictions worse [RES sections 5.11 to 5.12].
- No LLM that produces, adjusts or overrides a published number [RES section 7.6].
- No always-on server. No Glue or Athena before launch [STG].
- No mobile app in this scope. It is phase 2, built on the same published data [README].
- No compliance-mode Object Lock [STG decision 3].

5 Requirements

The repository has no separate requirements document, and the build plan that holds the original phase list is outside the repository. The requirements below are therefore written here from what the repository documents say the system does and must do. Each cites its source.

5.1 Functional requirements

ID	Requirement	Phase	Source
FR-01	Ingest 16 nflverse datasets as Parquet each day. Skip files whose source has not changed, retry transient failures, check each file is non-empty Parquet, and write a manifest.	1A	README
FR-02	Keep raw files as immutable dated snapshots. Cleaning happens in dbt, which reads the newest snapshot only.	1A	CLAUDE, STG
FR-03	Build staging, a player-week fact table, ranking models and a point-in-time feature store with dbt on DuckDB. Run all data tests (158) on every run, and fail the run on any test failure.	1A, 1C	README, NSF 9
FR-04	Rank QB, RB, WR, TE and K two ways: a composite score (20% efficiency, 80% production, percentile-scored within position) and season-to-date PPR points (standard scoring for kickers). Each week is a snapshot with rank movement.	1A	README, SEED
FR-05	List players below the minimum role without ranking them. The minimum scales with games the team has played.	1A	CLAUDE, SITE
FR-06	Rankings use only games through the ranked week. A test enforces it.	1A	README, CLAUDE
FR-07	Validate everything before publishing: schema, row counts, freshness. If anything fails, write nothing so the old data stays live. Write meta.json last.	1A	README, NSF 6, 7, 10
FR-08	Publish a versioned JSON contract (schema version 1). The site refuses data whose schema_version it does not understand.	1A, 1B	CON, WEB
FR-09	Project the coming week for each eligible player (at least 3 prior games and an average of at least 4 PPR points over the last five) with an 80% range. Drop players ruled Out or Doubtful. Show a Questionable player's score if he plays and the chance he does.	1C	RES 7.1, 7.2, README
FR-10	Build features only from games before the one being predicted. Tests change a future result and check earlier features do not move.	1C	RES 7.3, README
FR-11	Calibrate ranges on weeks the model never saw, and on the publishable population. The code refuses to calibrate on training rows.	1C	RES 5.16, 7.3
FR-12	Lock a week's projections before its first kickoff, write-once. Lock only when the first kickoff is under 24 hours and over 30 minutes away. Refuse, write nothing and exit non-zero once kickoff has passed with the week unlocked.	1C, 1D	OPS, NSF 18
FR-13	Re-locking a week with identical numbers is allowed (runs are retried). Re-locking with different numbers raises.	1D	STG decision 3
FR-14	Grade finished locked weeks against baselines (recent average, season average, last ten games). The report card shows MAE, RMSE, interval coverage and the margin, says so when nothing is graded yet, and shows losing weeks.	1C	RES 7.8, README
FR-15	Run a governed experiment loop. Write a failure report, accept one candidate feature, pre-check it, score it through the promotion gate, and append the verdict to the ledger whether it ships or not. Keep rejected candidates in the code.	1C	CLAUDE, RES 7.5, 7.6
FR-16	Backtest the rankings: does rank at week N predict week N+1? Choose settings on tuning seasons (2021 to 2023) and confirm on held-out seasons (2024, 2025). Write docs/backtest.md and backtest/summary.json, and publish the headline numbers in methodology.json.	1A	BT, STG
FR-17	Serve the site as a static export: home, rankings by position, player pages, projections by position, report card, methodology (rankings and predictions), about. Fetch data in the browser from /data/v1. Show loading, error and empty states, and a banner when published data is older than 36 hours.	1B	WEB, NSF 14, 16
FR-18	Run the daily task on AWS at 15:00 UTC: nfl-pipeline daily, which is run followed by predict --lock-when-due. If run fails, predict does not start.	1D	OPS
FR-19	Rebuild the warehouse from raw on every run and never read it back. Upload the finished file as a read-only snapshot so backtest and experiment can run as on-demand tasks. If the snapshot is missing, a reader rebuilds from raw.	1D	STG decision 1
FR-20	Define all infrastructure in Terraform. CI plans and never applies. Apply by hand.	1D	INF

ID	Requirement	Phase	Source
FR-21	CI runs on every push and pull request: lint, format check, tests with coverage, Docker build, web audit, lint, typecheck, unit and end-to-end tests, and Terraform fmt and validate. Terraform plan runs on pull requests that touch infra/. Merging to main deploys the image and the site.	1D	CI, INF
FR-22	Run backtest, experiment or any other command as an on-demand Fargate task with infra/run-task.ps1, on the same image.	1D	OPS, STG

5.2 Quality requirements

ID	Quality	Scenario and target	How verified	Source
QR-01	Cost	About a dollar a month on AWS. A $5 billing alarm exists. The pipeline makes no LLM API calls.	Billing alarm fired on purpose 2026-10-05.	README, NSF 1
QR-02	Freshness	Published data is refreshed daily. A game that kicked off more than 48 hours ago with no result, or a settled week missing from the rankings, blocks the publish. No successful run for 26 hours alarms. The site shows a banner at 36 hours.	Freshness gate tests; alarm fired by hand; e2e banner tests (appears at 37 h, absent at 35 h).	NSF 2, 6, 10, 14
QR-03	No silent failure	Every alarm uses treat-missing-data breaching unless a written reason says otherwise. Every alert path is fired once on purpose and the email is seen.	set-alarm-state on every alarm 2026-10-05; a real failing task also emailed.	NSF
QR-04	Record integrity	A locked forecast cannot be changed or deleted by the pipeline. Overrides are visible in CloudTrail.	Conditional write (412 on second write) plus versioning and Object Lock, governance mode, 365 days. Contract test on both storage backends.	STG decision 3
QR-05	No lookahead	Rankings and projections never use data from after the ranked or predicted game.	Ranking test; feature tests that change a future result.	README, RES 7.3
QR-06	Security	No stored AWS keys. Private buckets. CloudFront is the only reader of the site bucket. Least-privilege roles. Container runs non-root with HTTPS-only egress.	Terraform review; plan role cannot read data buckets; deploy role cannot touch IAM, billing, records or data/v1.	INF, DOCK, STG
QR-07	Reliability	A rerun is safe. Derived state is rebuilt from raw every run. A failed run publishes nothing, so visitors keep the last good data.	Rerun behavior; validation gate; OPS runbook.	STG, OPS
QR-08	Prediction accuracy	On held-out seasons, beat the best baseline by at least 0.15 MAE on the published population, keep 80% interval coverage within 78% to 82%, and show no leakage. The promotion gate for candidates keeps coverage within 0.75 to 0.85.	Walk-forward over 2021 to 2025. Final result: margin +0.170, coverage 0.797, 8,464 player-games.	RES 5.16, 7.5, 7.8
QR-09	Accuracy reporting	Every accuracy figure appears with its player count and is compared only with baselines scored on the same rows.	Report card and methodology pages.	RES 5.15, 7.8
QR-10	Accessibility	WCAG 2.2 A and AA, in both themes, with a keyboard-only journey. Movement is shown with arrows and words, never color alone. Phone layouts have no sideways scrolling.	axe checks on every page; mobile spec.	WEB
QR-11	Reproducibility	Research scripts and raw results are in research/. The same Docker image runs locally and on AWS.	research/*.py, research/results/*.json; Dockerfile.	RES 9, README
QR-12	Attribution	nflverse credited with CC BY 4.0. The site states it is independent of the NFL and is not betting advice.	Site footer and about page.	README, SITE

5.3 Standing project rules

These come from [CLAUDE] and [NSF]. They govern how changes are made.

ID	Rule
R-01	Read docs/storage-design.md before moving any file or changing config.py, storage.py or profiles.yml.
R-02	Raw files are immutable dated snapshots. dbt reads the newest only.
R-03	The published JSON is a contract. Changing it means bumping SCHEMA_VERSION, and web/lib/types.ts stays in step with contract.py.
R-04	Write tests alongside every change. No network in tests; use the fakes in pipeline/tests/conftest.py. Website tests do not depend on live data.
R-05	Only add --lock to predict before a week's first kickoff. A locked week cannot be rewritten.
R-06	Do not delete a rejected experiment candidate. The code is the record of what was tried.
R-07	Test it, do not inspect it: fire every new alarm once and see the email arrive.
R-08	Fail closed: treat a missing signal as broken. New alarms use treat_missing_data = breaching unless a reason is written.
R-09	Any new skip-because-nothing-changed optimisation gets a freshness assertion beside it.
R-10	Any new graceful fallback gets a log token and a counter.
R-11	Prefer raising to logging whenever the alternative is continuing with data that might be wrong.
R-12	Changes reach main by pull request only. test, web and infra must pass. Terraform is applied by hand.
R-13	Check docs/dependencies.md before bumping TypeScript, @types/node or ESLint.
R-14	Public text: first person singular, plain words, no em dashes, no filler words, no narrating code comments.

5.4 Phases, status and acceptance evidence

Phase	Scope	Status in the documents	Evidence
1A	Local pipeline: ingest, dbt with tests, rankings, published JSON, backtest	Done	README; BT generated 2026-10-06
1B	Website	Done; live	WEB; site reachable
1C	Weekly predictions, report card, experiment ledger	Built. No week had been graded when the README was last updated.	README (What is not done); RES
1D	AWS deployment	Running since 2026-10-05; every alert path fired on purpose	STG, NSF, OPS
1E	Launch polish	Named in CLAUDE.md. No status stated.	None in the documents read
2	Mobile app on the same published data	Planned, not started	README

5.5 Traceability

Component (section 7)	Requirements	Quality
7.1 Ingest	FR-01, FR-02	QR-02, QR-07
7.2 dbt models and ranking	FR-03, FR-04, FR-05, FR-06	QR-05
7.3 Validation gate and publish	FR-07, FR-08	QR-02, QR-07
7.4 Backtest	FR-16	QR-08, QR-11
7.5 Prediction engine	FR-09, FR-10, FR-11, FR-14	QR-05, QR-08, QR-09
7.6 Records and locking	FR-12, FR-13	QR-04
7.7 Experiment loop	FR-15	QR-08, QR-11
7.8 Website	FR-08, FR-17	QR-02, QR-10
7.9 Infrastructure and storage	FR-18, FR-19, FR-20, FR-22	QR-01, QR-04, QR-06
7.10 CI/CD	FR-21	QR-06
7.11 Monitoring and alarms	FR-12, FR-18	QR-02, QR-03

6 Architecture

6.1 System context (C4 level 1)

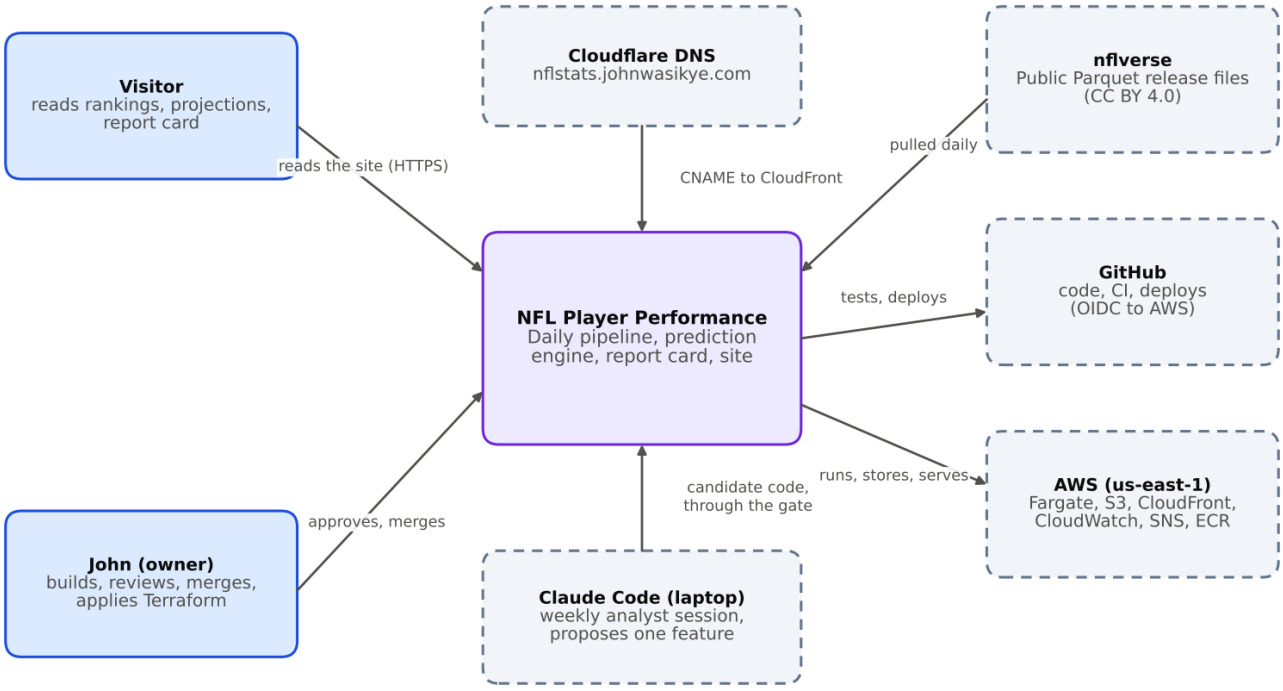


Figure 1. System context. Dashed boxes are external. The visitor reads the static site; John approves and merges; nflverse files are pulled daily.

6.2 Containers (C4 level 2)

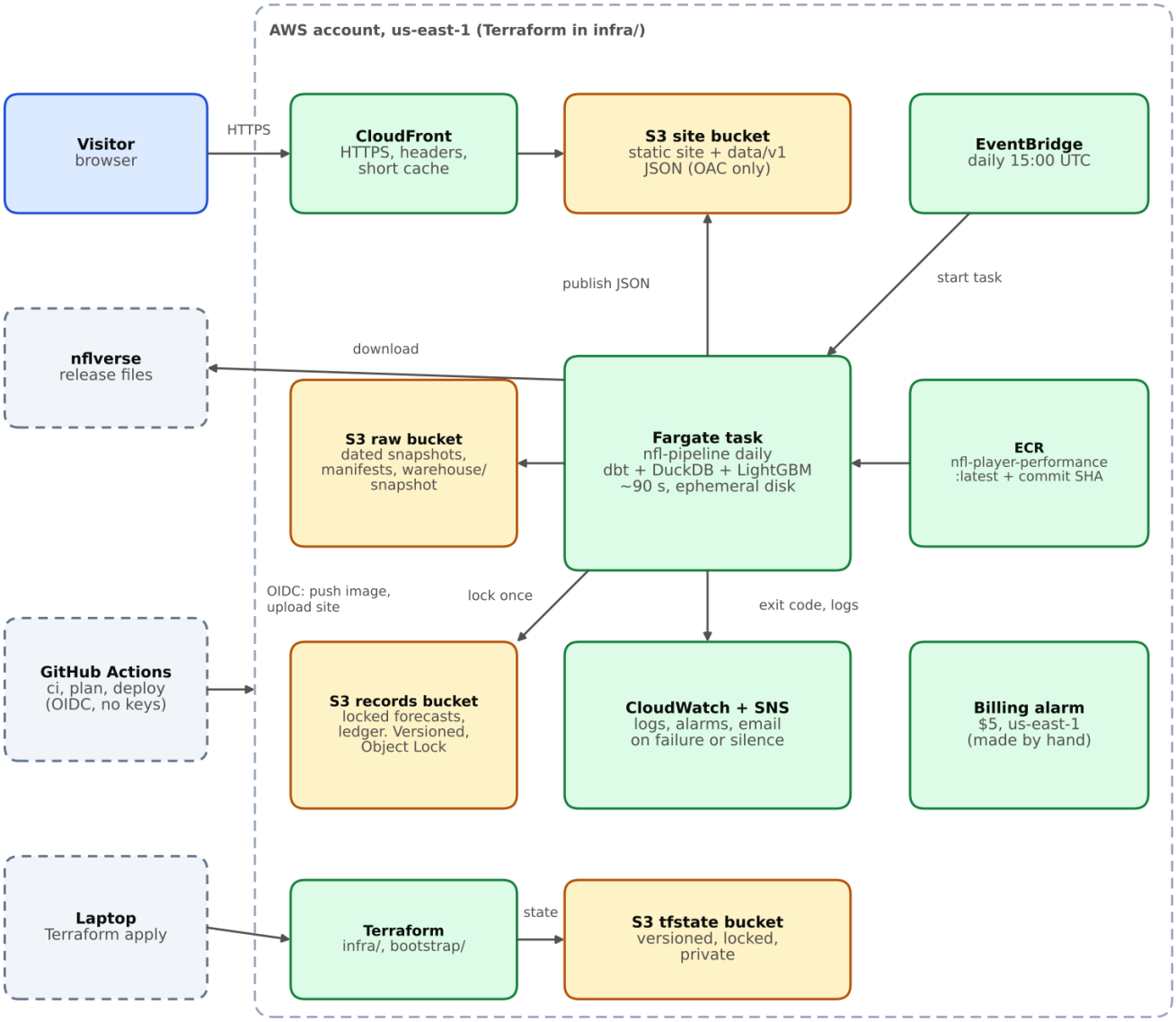


Figure 2. Containers inside the AWS account. The Fargate task is the only writer of derived data and the only writer of locked records. CloudFront is the only reader of the site bucket.

6.3 Components of the daily task (C4 level 3)

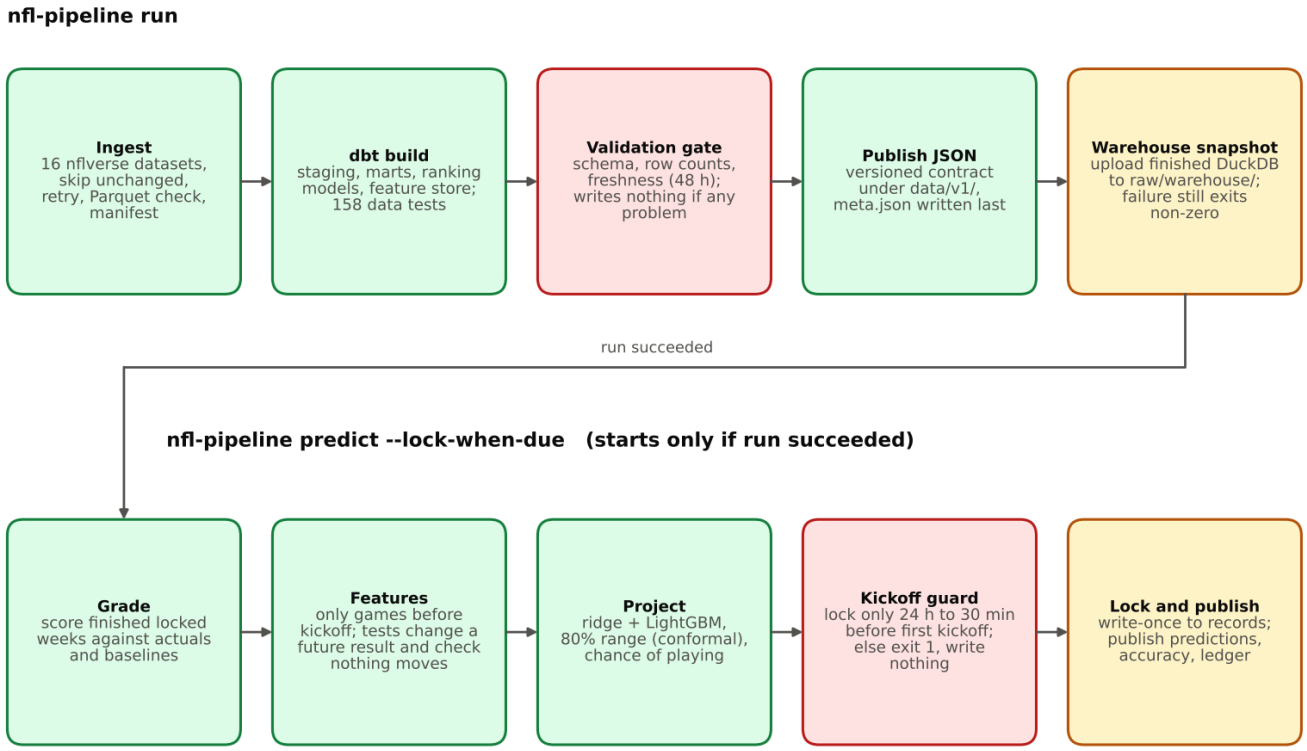


Figure 3. What nfl-pipeline daily does. The two red steps refuse and write nothing when their check fails. Predict starts only if run succeeded.

6.4 Repository structure

```
pipeline/nfl_pipeline/   ingest, storage, publish, contract (published schema), predictions, CLI
pipeline/tests/          unit tests and dbt integration tests (no network)
dbt/                     staging, marts, ranking and feature models, seeds, data tests
web/                     the website (Next.js static export): pages, components, tests
infra/                   Terraform: buckets, ECR, ECS, schedule, alarms, CloudFront, domain, GitHub OIDC
research/                the studies behind the prediction design, with results
docs/                    operations runbook, design notes, the backtest, the research paper, dependencies
.github/workflows/       CI, Terraform plan on pull requests, deploy on merge
.claude/                 Claude Code project folder (contents not read)
data/predictions/        seed copy of the ledger (S3 is the runtime store)
```

The ranking method lives in dbt/seeds/ranking_config.csv and ranking_weights.csv. Change a seed and rebuild to try another setup [README].

7 Component breakdown

Names of commands, files and settings in this section are the ones in the repository. Behavior is described from the repository documents; the source files themselves were not read (see the note after the source key).

7.1 Ingest: pipeline/nfl_pipeline/

Job: download the nflverse release files and keep them as immutable dated snapshots [FR-01, FR-02].

- **Inputs:** 16 nflverse datasets published as Parquet (schedules, players, weekly player stats, play-by-play, injuries, snap counts, rosters, depth charts, participation, advanced stats, charting data, Next Gen Stats). The weekly stats file is named stats_player_week_<season>; the regpost file is season totals with no week [CLAUDE].

- **Outputs:** dated raw snapshots and a manifest, under ./data/ locally or in the raw bucket on AWS.
- **Behavior:** skips files whose source has not changed, retries transient failures, checks that each file is non-empty Parquet, and writes a manifest [README]. The skip logic is paired with a freshness check in the validation gate, so a wrongly skipped file cannot publish last week's numbers as current [NSF 6].
- **Options** for run and ingest: --seasons 2026 or --seasons 2021-2025, --datasets a,b, --force [README].
- **Storage abstraction:** a Storage protocol with put_bytes and get_bytes, extended with put_bytes_if_absent(key, data) -> bool (S3 passes IfNoneMatch="*" and maps a 412 to False; local uses O_CREAT | O_EXCL, never os.replace) and list_keys(prefix) -> list[str]. LocalStorage and S3Storage implement it; MinIO stands in for S3 locally [STG decision 5].
- **Config:** config.py holds s3_records_bucket and ledger_key. build_records_storage() returns LocalStorage rooted at <data>/predictions locally, or S3Storage on the records bucket. build_site_storage() returns the site bucket on S3 or <data>/site locally, so key layouts are identical in both places [STG].

7.2 Transform: dbt/ on DuckDB

Job: clean and model the data, rank players, and build a point-in-time feature store [FR-03 to FR-06].

- **Layers:** staging (standardized teams and positions), a player-week fact table, ranking models, and the feature store used by predictions. Every model is materialized as a table and nothing is incremental, so dbt build recreates the whole graph from raw plus seeds [README, STG].
- **Reading raw:** the latest_raw macro reads read_parquet('<raw_root>/<dataset>/**/*.parquet', filename = true) and keeps only the newest snapshot. On AWS, RAW_ROOT is s3://<raw-bucket>/raw and DuckDB's httpfs extension reads S3 directly, so raw never lands on the task's disk [STG decision 2].
- **Targets:** dev (local files), aws (a secrets entry of type s3 with provider credential_chain, so the Fargate task role supplies credentials), and minio (explicit key, endpoint, path-style, no SSL) [STG].
- **Tests:** 158 data tests run on every run, including tests that prove a feature never sees its own week. A drop in the test count is treated as a defect [README, NSF 9].
- **Seeds:** after changing a seed's columns, run dbt seed --full-refresh or the old columns stay [CLAUDE].

Ranking method

Each metric is percentile-scored among ranked players at the same position for that week. Percentiles are averaged into an efficiency score and a production score (0 to 100), then blended 20% efficiency and 80% production. The second ranking is season-to-date PPR fantasy points; nflverse does not score kickers, so they get common standard scoring [SITE, SEED]. Rankings are as of week N with no lookahead, and a test enforces it. Players below the minimum role are listed but not ranked; the minimum scales with games the team has played [CLAUDE].

Position	Minimum role (min_role_per_week)	Efficiency weight	Production weight
QB	14	0.2	0.8
RB	6	0.2	0.8
WR	3	0.2	0.8
TE	2.5	0.2	0.8
K	2.5	0.2	0.8

The metrics behind each position are listed in appendix A.

7.3 Validation gate and publish

Job: refuse to publish anything wrong, and write a versioned JSON contract the site reads [FR-07, FR-08].

- **Checks:** schema validation against the pydantic models in contract.py (extra fields forbidden), row counts (MIN_RANKED per position), and freshness. If any check fails nothing is written and the previously published data stays live [README, NSF 7].

- **Freshness (_freshness_problems):** check A fails if a regular-season game kicked off more than STALE_AFTER_HOURS (48) ago and still has no result in stg_schedules. Check B fails if a week settled longer than the threshold ago is not covered by the rankings. A game with neither kickoff nor date is reported, not skipped, and a missing stg_schedules is a failure, not a pass [NSF].
- **Timezone handling:** kickoff_et is a naive Eastern timestamp. The freshness check uses a fixed 5-hour offset (EASTERN_TO_UTC_HOURS), which can only make a game look up to an hour more recent and so delay a complaint, never cause a false alarm [NSF].
- **Output:** JSON under data/v1/ (locally data/site/data/v1/). meta.json is written last, so it only ever points at a complete set of files [CON].
- **Snapshot:** after the build, snapshot.py uploads the warehouse to raw/warehouse/warehouse.duckdb. A fallback logs WAREHOUSE_SNAPSHOT_FALLBACK; a failed upload logs WAREHOUSE_SNAPSHOT_UPLOAD_FAILED, still publishes, then exits non-zero [NSF 13].

7.4 Backtest

Job: test whether a player's rank at week N predicts what he does in week N+1 over past seasons (nfl-pipeline backtest) [FR-16]. The efficiency weight was chosen on tuning seasons 2021 to 2023 and checked on held-out seasons 2024 and 2025. The selection rule picks the highest efficiency weight whose average Spearman across positions on the tuning seasons is within 0.005 of the best, so near-ties go to more efficiency [BT].

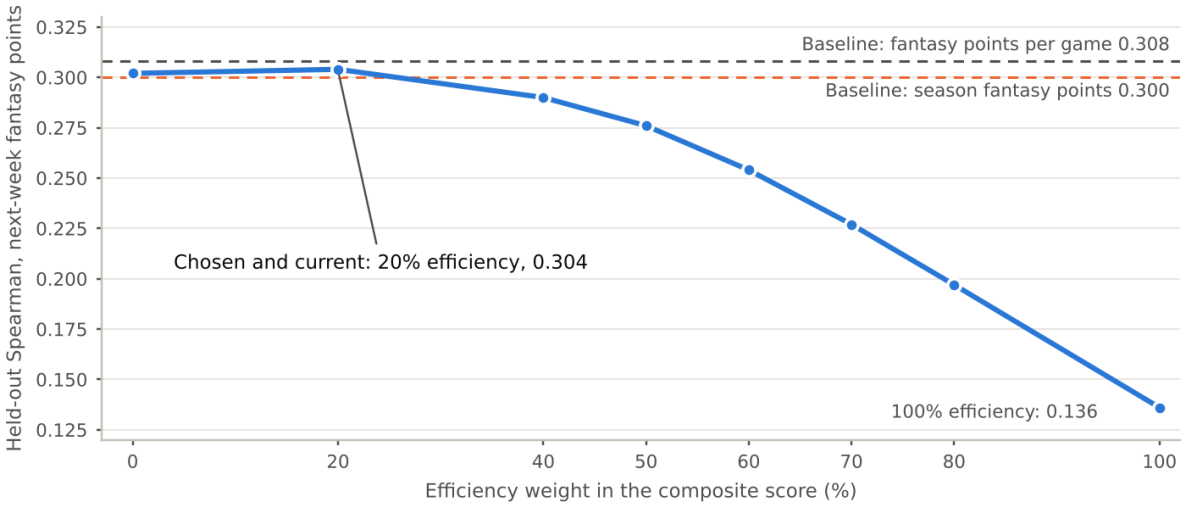


Figure 4. Held-out Spearman between rank at week N and next-week fantasy points, by efficiency weight. The chosen 20% ties the baselines. Heavy efficiency weights predict clearly worse. Data from [BT].

In short: a mostly production-based composite roughly ties ranking by fantasy points per game, and ranking with a high efficiency weight is noticeably worse. Held-out Spearman for the chosen 20% is 0.304 against 0.308 for points per game and 0.300 for season points. By position the composite is ahead for WR (+0.016) and TE (+0.023) and behind for RB (-0.025), K (-0.026) and QB (-0.009, within noise). Full tables are in appendix B.

7.5 Prediction engine: pipeline/nfl_pipeline/predict/

Job: project each eligible player's next game with a range, and grade it afterwards [FR-09 to FR-11, FR-14].

What is predicted, and for whom

- **Target:** PPR fantasy points (standard scoring for kickers) with an 80% range and, for a player who might not play, the chance he takes the field [RES 7.1].
- **Eligibility:** a player is projected if he has at least 3 prior games and averaged at least 4 PPR points over his last five. Below that line the model was measured losing to the player's own recent average. That is roughly 280 players a week across five positions [RES 7.1, 5.15].
- **Three populations, deliberately different:** train on every player-week with at least 3 prior games; calibrate ranges on the publishable population only; publish only the eligible players [RES 5.16].
- **Availability:** players ruled Out or Doubtful are dropped, not shown as zero. Questionable players show a projection for if they play and a separate chance that they do (base rate 63%, far lower for quarterbacks), estimated from practice participation. Late scratches are not graded against the points model [RES 7.2].

Model

- **Point estimate:** the average of a ridge regression and a LightGBM model.
- **80% range:** two further LightGBM models fitted to the 10th and 90th percentiles, widened by a split-conformal correction measured on weeks the quantile models never saw. The code refuses to calibrate on rows the model was trained on [RES 7.3].
- **Features (six groups):** recent form, usage, advanced-stats and Next Gen Stats measures, game context (implied total, spread, home, rest, week), weather, and the opponent's points allowed to the position. Every feature uses only games before the one predicted, with tests that change a future week's result and check earlier features do not move [RES 7.3].
- **Home and away:** a feature in the model, so the league-wide effect is learned from data (about +5% of points, quarterbacks about +8%). No stadium or team term [RES 5.1, 7.4].
- **Cadence:** weekly retraining is kept (worth about 0.7% and it does not compound), and ranges are recalibrated each time projections are generated [RES 5.11, 7.5].
- **Known weakness:** the schedule records wind and temperature only after a game is played, and no forecast source is connected, so those two features take typical values for games that have not happened [RES 7.3].

Why the population restriction exists

The first full walk-forward run beat the best baseline by only 0.052 MAE, short of the 0.15 bar. Splitting by recent scoring showed the model losing to a plain average on low-volume players and winning clearly on players with a real role. Restricting to players averaging at least 4 points cleared the bar. Training on everyone but calibrating on the published population restored interval coverage to 0.797 with identical MAE. None of the final gain came from a better model [RES 5.15, 5.16].

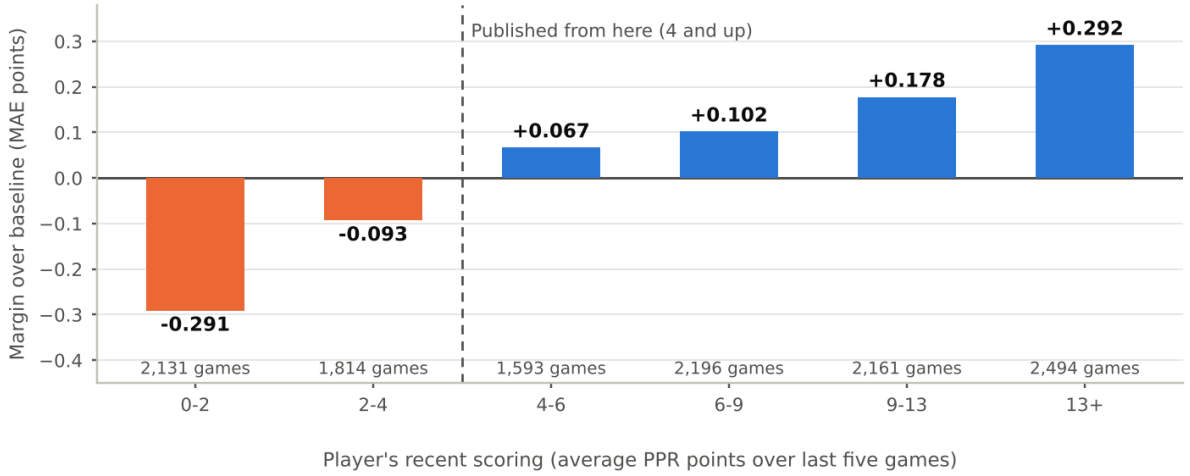


Figure 5. Margin over the best baseline by recent scoring tier, held-out walk-forward. Orange bars are tiers where the model lost to the baseline. Data from [RES 5.15].

Eligible if recent average at least	Player-games	MAE	Baseline	Margin	Coverage	Clears 0.15
0	12,388	4.463	4.515	+0.052	0.797	no
2	10,278	4.931	5.047	+0.116	0.797	no
4 (chosen)	8,464	5.278	5.436	+0.159	0.798	yes
6	6,879	5.586	5.752	+0.166	0.799	yes
8	5,421	5.850	6.018	+0.168	0.801	yes
10	4,037	6.136	6.346	+0.210	0.798	yes

Raw MAE rises with the cut-off because higher-scoring players are more variable. Each row is a different population, so only the margin within a row can be compared. The final configuration scores MAE 5.266 against a best baseline of 5.436: margin +0.170, coverage 0.797, 8,464 player-games [RES 5.16].

Limits [RES 7.9]: the ceiling is about 0.29 R-squared for fantasy points and 0.05 to 0.09 for touchdowns, so week-to-week accuracy will wobble. Early in a season the model has little recent form and its margin is weakest. Late injuries and coaching decisions can invalidate a locked forecast.

7.6 Records and locking

Job: keep the only two records, locked forecasts and the experiment ledger, unchangeable after the fact [FR-12, FR-13].

Every piece of state is either **derived** (reproducible from code plus raw data, disposable, needs no protection) or a **record** (its value is that it cannot change after the fact, so it needs an integrity mechanism, not a convention). A graded forecast that a later run can rewrite is not evidence [STG].

State	Size (2026-10-01)	Class	Home on AWS
Raw nflverse snapshots	140 MB	Source of truth, immutable, dated	raw bucket, versioned; noncurrent versions expire after 30 days; current snapshots are kept
warehouse.duckdb	163 MB	Derived	Rebuilt on the task's disk each run, then published to raw/warehouse/ as a read-only snapshot
Published JSON	1.6 MB	Derived, serves the site	site bucket under data/v1/
Locked weekly predictions	about 60 KB a week	**Record**	records bucket, write-once
Experiment ledger	3.3 KB	**Record**	records bucket, plus a git mirror (not built yet)
dbt seeds (ranking config)	tiny	Code	git, baked into the image
Backtest summary	4 KB	Derived	raw bucket at backtest/summary.json; headline numbers in methodology.json

Write-once is enforced, not assumed

1. **Conditional write.** PutObject with If-None-Match: "*" creates the object only if the key is absent and returns 412 Precondition Failed otherwise. It is atomic and prevents a second version from ever being created.
2. **Versioning plus Object Lock, governance mode,** with 365 days of retention on every version written. This prevents the first version from being deleted or expired.

They are not redundant: Object Lock protects a version and does not stop new versions being written, while a conditional write does not stop a delete. Overriding governance needs s3:BypassGovernanceRetention plus an explicit header, and the override appears in CloudTrail, which makes tampering evident. On a 412 the code fetches the stored object and compares: identical numbers log and return it, different numbers raise. Object Lock is enabled when Terraform creates the bucket, because retrofitting it is awkward [STG decision 3, NSF 11].

The kickoff guard (predict --lock-when-due)

- Locks a week only when its first kickoff is more than 30 minutes and less than 24 hours away.
- If the week is already locked, it publishes the existing lock and never recomputes it.
- If kickoff has passed with the week unlocked, or the kickoff cannot be established, it exits 1 and writes nothing. The task-failed rule emails.
- While the next kickoff is more than a day away and no players can be projected (the roster is not published yet), it notes the gap and exits 0. Once the lock window opens or has passed, it raises and exits 1.
- The kickoff estimate errs early (UTC-4 all year), so doubt can only refuse. 25 tests cover it, and the 5 that assert a refusal were each seen to fail with the guard disabled [NSF 18, 19].

Why it exists: without it, a late scheduled run would stamp a forecast made during the games as written before them, and the report card's central claim would be false with nothing erroring.

7.7 Experiment loop

Job: improve the model only through changes that earn their place on held-out data [FR-15].

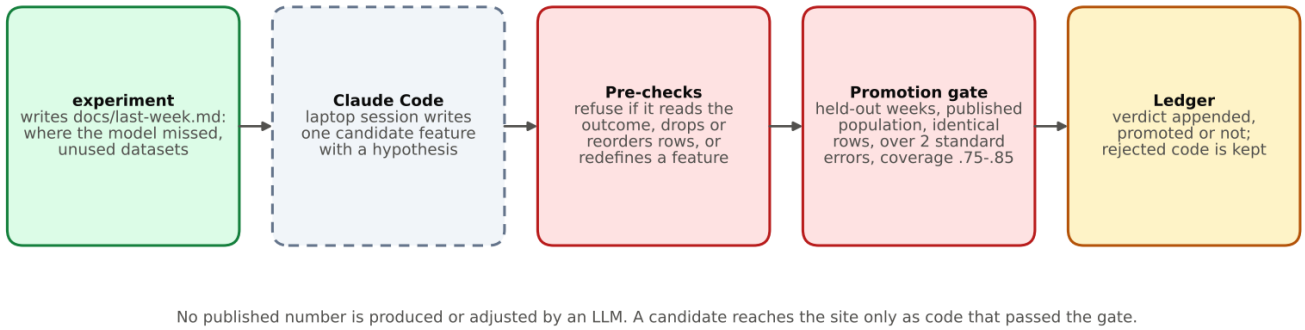


Figure 6. The weekly experiment loop. Claude proposes code; the gate decides; the ledger records the verdict either way.

- **nfl-pipeline experiment** writes docs/last-week.md: where the model missed most recently, split by position and by recent scoring, and which datasets are still unused [CLAUDE, RES 7.6].
- John gives the report to Claude Code, which writes one candidate feature with a stated hypothesis into pipeline/nfl_pipeline/predict/proposals.py. **nfl-pipeline experiment --run <n>** puts it through the gate.
- **Pre-checks:** a candidate is refused before scoring if it reads the outcome, drops or reorders rows, or redefines an existing feature. The outcome check runs the candidate a second time with the answer blanked and compares [RES 7.6].
- **Promotion gate:** the candidate must beat the current model on weeks neither was trained on, by more than the noise, and keep interval coverage inside 0.75 to 0.85. Champion and challenger are scored on identical rows, so the per-row difference in absolute error has a standard error, and a change under two standard errors from zero is rejected however large the raw margin looks. Candidates are scored on the population the site publishes [RES 7.5].
- **Ledger:** every verdict is appended, promoted or not. Each entry holds entry_id, proposed_at, hypothesis, change, champion_score, challenger_score, improvement, promoted, reason and metric (mae or margin_over_baseline) [CON]. Rejected code is never deleted [CLAUDE].
- **State of the ledger** when the research paper was written: four entries, the three population experiments and one feature candidate (ratio of recent form to season average) that was rejected at 0.08 standard errors with 49.99% of rows improved [RES 7.5]. The ledger in git is only a seed; the live ledger is in the records bucket.
- **Evidence base:** LLM-driven feature engineering (CAAFE, Hollmann et al., NeurIPS 2023) improved 11 of 14 tabular datasets, mean ROC AUC 0.798 to 0.822. The evidence rules out an LLM producing or adjusting numbers [RES 1, 7.6].

- **Cost:** the pipeline makes no API calls. One short weekly session fits inside the Claude Pro plan. If a session never happens, predictions still publish and the ledger does not advance that week [RES 7.6].

7.8 Website: web/

Job: show rankings, projections, the report card and the methodology, with data fetched in the browser [FR-08, FR-17].

Route	What it shows
/	Season and week status, biggest movers, the top five at each position
/rankings/[position]/	Sortable, filterable table for QB, RB, WR, TE or K; composite or fantasy view; week selector
/player/?id=...	Rank history chart, stat tiles and why the player is ranked where they are. One static page serves every player.
/predictions/[position]/	Next week's projection for each player with the range it could land in
/report-card/	Locked forecasts graded against what happened, and the log of every experiment tried
/methodology/ and /methodology/predictions/	The exact settings in use and the backtest; how projections are made and the research paper behind them
/about/	Data sources, attribution, glossary, disclaimers

- **Stack:** Next.js static export, TypeScript, Tailwind CSS. No UI or chart library; components and SVG charts are hand-written. Light and dark themes from CSS variables, with a saved choice applied before first paint [WEB].
- **Fails visibly:** loading skeletons, an error state with retry, an empty state, and a banner if published data is over 36 hours old, driven by the meta.json timestamp [WEB, NSF 14].
- **Contract:** lib/types.ts mirrors contract.py. The site refuses data whose schema_version it does not understand. NEXT_PUBLIC_DATA_BASE points the site at other data, for example a CloudFront address [WEB].
- **Deploy:** merging to main builds the site and runs scripts/deploy-site.sh, which uploads out/ to the site bucket and refreshes CloudFront. It never touches data/v1/, which the daily pipeline owns [WEB].
- **Avatars** are initials on team colors. No photos or logos [CLAUDE].

7.9 Infrastructure and storage: infra/

Job: define every AWS resource as code [FR-18 to FR-20, FR-22].

Bucket (prefix jw-nfl-player-stats-)	Contents	Policy
raw	Raw snapshots, ingest manifests, the warehouse/ snapshot, backtest summary	Versioned. Noncurrent versions expire after 30 days. Current raw snapshots are never expired.
records	Locked predictions and the ledger	Versioned. Object Lock, governance mode, 365 days on every version. No expiry. Tiny.
site	The static site and published JSON under data/v1/	Versioned. Noncurrent versions expire after 14 days. CloudFront OAC is the only reader.
tfstate	Terraform state	Versioned, encrypted, private, Terraform-native locking. Created by infra/bootstrap/ with local state.

- **Compute:** an ECS cluster (nfl-player-performance) and task definition (nfl-pipeline) on Fargate, with 20 GB of ephemeral storage by default. The image is in the ECR repository nfl-player-performance with tags latest and the commit SHA [OPS, STG].
- **Schedule:** an EventBridge rule starts the task daily at 15:00 UTC.
- **CDN and DNS:** CloudFront (distribution E1OZ5WDKRFMJ25) in front of the private site bucket, HTTPS, security headers, short caching for the daily data. The certificate for nflstats.johnwasikye.com is in domain.tf. DNS lives at Cloudflare, so two records are added by hand, both DNS only: the certificate validation record and a CNAME from nflstats to the distribution [INF].

- **Terraform files:** bootstrap/, buckets.tf, ecs.tf, monitoring.tf, cloudfront.tf, domain.tf, github.tf, run-task.ps1. The code names no AWS profile, so CI and the laptop run the same files [INF].
- **Encryption:** SSE-S3 (free). A customer-managed KMS key would cost about $1 a month plus request charges and earns its place only on the records bucket, because key use shows in CloudTrail. SSE-S3 is the documented setting; whether the records bucket also got a KMS key is not stated (OQ-9) [STG].
- **Image (Dockerfile):** python:3.12-slim in two stages; installs libgomp1 for LightGBM; runs as a non-root user (uid 1000); pre-installs DuckDB's httpfs extension because the extension downloads over plain HTTP, which the task's HTTPS-only security group would block at run time; the dbt project is copied in; dbt build output goes to /tmp; entrypoint nfl-pipeline [DOCK].
- **Dependencies [PYP]:** boto3, dbt-duckdb, duckdb, httpx, lightgbm, numpy, pandas, pyarrow, pydantic, scikit-learn, scipy; dev: pytest, pytest-cov, ruff (line length 100, target py310).

7.10 CI/CD: .github/workflows/

Workflow	Runs when	What it does	AWS role
ci.yml	Every push to main and every pull request	Job test: ruff check, ruff format --check, pytest with coverage, Docker build. Job web: npm ci, npm audit --omit=dev --audit-level=high, lint, typecheck, unit tests, Playwright end-to-end. Job infra: terraform fmt -check, init -backend=false, validate (also for bootstrap).	None; no AWS access needed
plan.yml	Pull requests that touch infra/	terraform plan, shown on the run summary	github-plan: read-only, cannot change anything, cannot read the data buckets
deploy.yml	Merge to main	Pushes the image (latest and the commit SHA) and uploads the site	github-deploy: cannot change infrastructure, touch IAM or billing, read the records, or overwrite data/v1/

GitHub reaches AWS through OIDC: each run presents a short-lived signed token and AWS checks that it names this repository and the expected ref. main is protected: changes arrive by pull request and test, web and infra must pass. Actions are pinned by full commit SHA with the version in a comment, and Dependabot proposes updates weekly [INF, CI, DEP].

7.11 Monitoring and alarms

Job: make every failure loud, and make silence an alarm [QR-03].

Signal	Mechanism	Notes
Task ran and exited non-zero	ECS task-state-change rule, email via SNS topic nfl-pipeline-alerts	Email names stopCode, for example EssentialContainerExited
Task never started (image pull, role, networking)	The same ECS rule (email names TaskFailedToStart) plus an EventBridge FailedInvocations alarm	Your own code never runs, so no alarm based on its output could fire
No success in 26 hours	Alarm nfl-pipeline-no-success-in-26h on the rule's TriggeredRules metric, treat-missing-data breaching	The keystone: catches a pipeline that dies completely and stops emitting
EventBridge cannot deliver	Alarms nfl-pipeline-rule-delivery-failed-*	
Warehouse snapshot fallback or failed upload	Log metric filters on WAREHOUSE_SNAPSHOT_FALLBACK and WAREHOUSE_SNAPSHOT_UPLOAD_FAILED; alarms nfl-pipeline-fallback and nfl-pipeline-upload_failed (treat-missing notBreaching)	No event is the healthy state here, the opposite of the staleness alarm
Spend	Billing alarm Bill_Over_5_Dollars in us-east-1, SNS topic Billing_Alerts	Billing metrics publish only in us-east-1. Made by hand.

Signal	Mechanism	Notes
Stale published data	Validation gate refuses; site banner at 36 hours; no-success alarm at 26 hours	A refused publish exits non-zero, so it never counts as a success
A forecast locked late	Kickoff guard exits 1 and writes nothing; task-failed rule emails	Seen working on AWS 2026-10-05: week 4, never locked, was refused

Every alarm was set to ALARM by hand once and its email arrived, both SNS subscriptions are confirmed, ActionsEnabled is set explicitly, and a real failing task also emailed [NSF 1 to 5]. The log-token path for a real snapshot fallback has not been exercised (OQ-8).

8 Runtime view

8.1 Daily run (15:00 UTC)

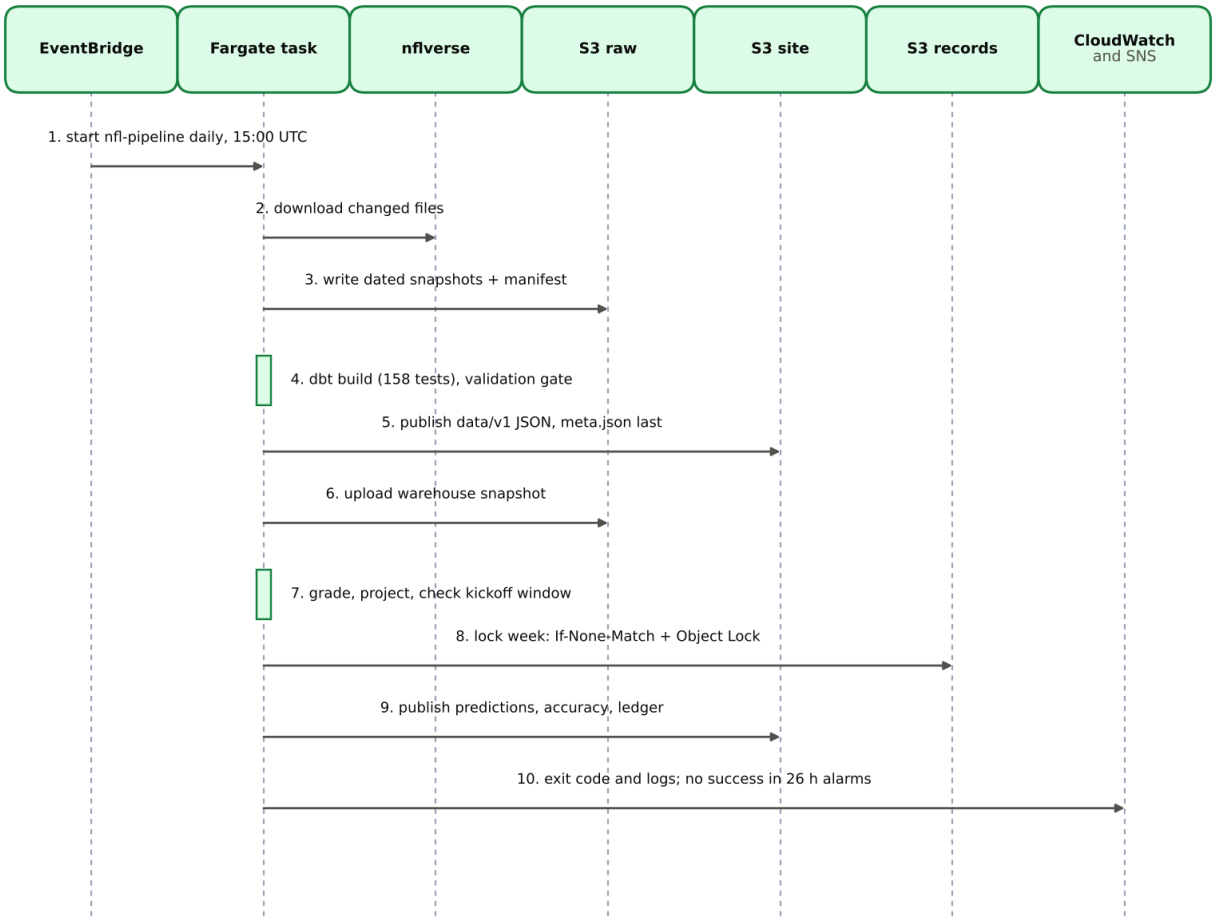


Figure 7. One daily run, from the schedule to the alarms.

1 EventBridge starts one Fargate task running nfl-pipeline daily [OPS].

2 Ingest downloads changed nflverse files and writes dated snapshots and a manifest.

3 dbt rebuilds the warehouse from raw on the task's disk and runs all 158 tests. A failed test stops the run: *run stopped: dbt build failed, so nothing was published*. The old data stays live.

4 The validation gate checks schema, row counts and freshness. A failure logs *validation failed with N problem(s); nothing was published* and the old data stays live.

5 Publish writes versioned JSON to data/v1/ with meta.json last. The finished warehouse is uploaded as a read-only snapshot.

6 predict --lock-when-due grades finished locked weeks, projects the coming week, and checks the kickoff window. It does not start if step 2 to 5 failed.

7 If the window is open, the week is locked with a conditional write. Re-locking identical numbers is fine; different numbers raise and need a person.

8 Predictions, accuracy and the ledger are published.

9 The task exits. A non-zero exit emails. No successful run in 26 hours alarms.

The task takes about 90 seconds and costs a few cents [OPS].

8.2 Public request

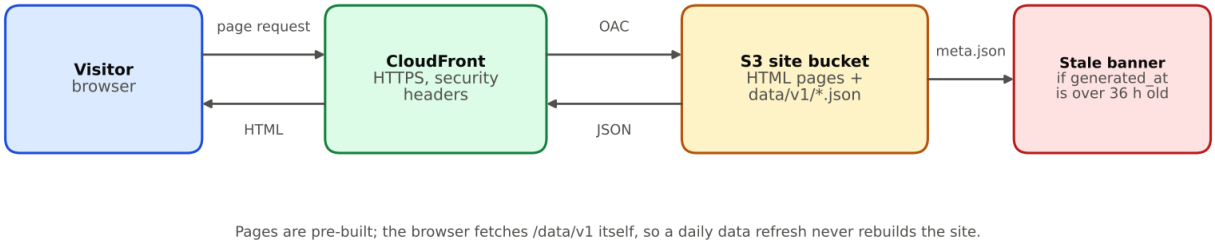


Figure 8. A visitor's request. Pages are pre-built; the browser fetches /data/v1 itself.

The site works without any server of its own. A daily data refresh never needs a site rebuild, because the pipeline owns data/v1/ and the deploy script never touches it [WEB].

8.3 Deploy flow

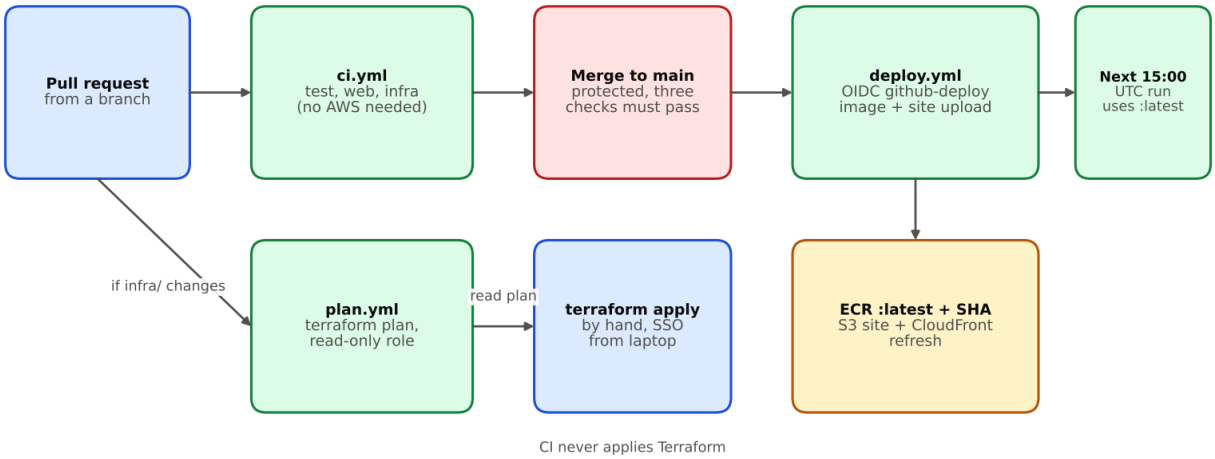


Figure 9. From pull request to the next scheduled run. Terraform is applied by hand after reading the plan.

After merging a change to the image, the next scheduled run uses it. To publish sooner, run infra/run-task.ps1 daily. CloudFront caches pages for a few minutes; clear it with an invalidation if a page looks stale after a deploy [OPS].

8.4 Weekly cycle

- **Lock timing.** A week is locked once, on the run that falls inside its window. For a Thursday night opener that is Thursday's 15:00 UTC run, about nine hours before kickoff. For a first game on Sunday at 1:00 PM Eastern it is Sunday's run, and Saturday's if the first game is the early London kickoff [OPS].
- **Grading.** Each run grades any locked week that has finished. Until a locked week finishes, the report card says there is nothing to grade yet [README].
- **Roster gap.** The player list for a week comes from nflverse's weekly roster file. Between one week ending and the next roster appearing, nothing can be projected (OQ-1) [OPS].
- **Analyst session.** Once a week, John runs the experiment report and a Claude Code session on the laptop (section 7.7).
- **Clock change.** The 15:00 UTC schedule is 10:00 AM Eastern after the clocks change on 2026-11-01 [OPS].

9 Data model

9.1 Published files

The website, and later the mobile app, read only these JSON files. Every file is built from the pydantic models in contract.py and validated before it is written [CON].

Path under data/v1/	Model	Contents
meta.json	Meta	schema_version, generated_at, pipeline_version, data_as_of (nflverse dataset to update time), season, latest_week, week_complete, weeks, positions. Written last.
methodology.json	Methodology	Per-position settings (minimum role, weights, metrics), kicker scoring, backtest headline numbers
rankings/{season}/{week}/{position}.json	RankingsFile	Ranked players first, then unranked. Each has composite and fantasy views.
players/{player_id}.json	PlayerFile	Rank history this season and the latest score breakdown
movers/{season}/{week}.json	MoversFile	Biggest composite-rank risers and fallers per position

Path under data/v1/	Model	Contents
predictions/{season}/{week}/{position}.json	PredictionsFile	Projections with ranges, status preliminary or locked
predictions/latest.json	PredictionsIndex	Which week the site should show, its status and lock time
accuracy/{season}.json	AccuracyFile	Graded weeks, season to date, a plain-language verdict built only from the numbers
ledger.json	LedgerFile	Every experiment, including failures

predictions/latest.json is a separate small file because rankings and predictions are published by different commands and are routinely about different weeks: rankings describe the last week played, predictions the next one. Folding both into meta.json would let either command publish a meta.json that contradicted the other [CON].

9.2 Contract models (selected fields)

Model	Fields
RankedPlayer	player_id, name, team, injury_status, games, qualified, composite, fantasy
CompositeView	score, efficiency, production (0 to 100), rank, rank_prev, movement (previous rank minus current; positive is up), is_new
FantasyView	points, per_game, rank, rank_prev, movement, is_new
BreakdownItem	metric, component (efficiency or production), value, percentile, weight, contribution_points
PredictedPlayer	player_id, name, team, opponent, is_home, position, points, low, high, probability_of_playing, expected_points, injury_status, status (preliminary or locked), locked_at
GradedWeek	season, week, player_games, mae, rmse, interval_coverage, baseline_mae (by baseline), frozen_model_mae (null until a frozen model exists)
LedgerEntry	entry_id, proposed_at, hypothesis, change, champion_score, challenger_score, improvement, promoted, reason, metric (mae or margin_over_baseline)
PositionMethodology	position, min_role_per_week, efficiency_weight, production_weight, metrics (metric, component, direction, weight)

All models forbid extra fields. Changing a model is a deliberate edit to contract.py, and a breaking change bumps SCHEMA_VERSION (currently 1) [CON].

9.3 Warehouse

The DuckDB warehouse (warehouse.duckdb, 163 MB) holds staging models, the player-week fact table, ranking models and the point-in-time feature store. It is derived and disposable: rebuilt from raw every run and never read back [STG]. Raw data and the warehouse are not part of the public contract.

10 API

There is no application server. The published JSON files in section 9.1 are the API: static objects in the site bucket served by CloudFront under /data/v1/, read by the browser. The version is in the path (v1) and in every file's schema_version. A breaking change bumps SCHEMA_VERSION; the site refuses a version it does not understand [CON, WEB].

- **Caching:** CloudFront uses short caching for the daily data and security headers on responses [OPS, README].
- **Consistency:** meta.json is written last, so a reader that loads it sees a complete set of files [CON].
- **Consumers:** the website now; the planned mobile app later, on the same files [README].
- **Not provided:** an OpenAPI description, authentication, rate limiting. Nothing in the documents read defines them.

11 Deployment view

What	Runs where	Notes
Daily pipeline and predictions	Fargate task nfl-pipeline in cluster nfl-player-performance, us-east-1	Started by EventBridge at 15:00 UTC. About 90 seconds. Image from ECR (latest and commit SHA).
Backtest, experiment, ad hoc commands	The same Fargate task with a different command	infra/run-task.ps1 starts it, waits, prints how it ended. They download the warehouse snapshot, or rebuild from raw if it is missing.
Weekly analyst session	John's laptop, Claude Code, inside the Pro plan	The only thing that still needs the laptop. Moving it to AWS (Claude Code on Amazon Bedrock) is researched and not built.
Site	S3 site bucket behind CloudFront, domain nflstats.johnwasikye.com (DNS at Cloudflare)	Static export. Deployed by deploy.yml on merge.
Published data	S3 site bucket, data/v1/	Owned by the daily pipeline. Never overwritten by the site deploy.
Records	S3 records bucket	Object Lock, governance mode, 365 days.
Alerts	CloudWatch alarms and SNS topics nfl-pipeline-alerts and Billing_Alerts	Email to the address set in infra/monitoring.tf.
Infrastructure	Terraform in infra/ and infra/bootstrap/; state in the tfstate bucket	Applied by hand from the laptop with SSO profile nfl_player_stats. CI plans only.
Local development	Laptop: Python venv, or Docker (pipeline-local), plus a MinIO service	Everything lands in ./data/. MinIO console at localhost:9001 (local only).
CI	GitHub-hosted ubuntu-24.04 runners	ci.yml, plan.yml, deploy.yml.

Environments: the documents describe one AWS environment. Local runs use ./data/ and MinIO as a stand-in for S3, selected by the dbt target (dev, aws, minio). No separate dev copy of the AWS infrastructure is described.
Schedules: daily 15:00 UTC run; Dependabot weekly; CI on every push and pull request; the analyst session weekly and by hand.

12 Cross-cutting concerns

12.1 Security

- **No stored AWS keys.** GitHub Actions uses OIDC: each run presents a short-lived signed token and AWS checks that it names this repository and the expected ref. The Fargate task uses its task role, and DuckDB reads S3 through the credential chain [INF, STG].
- **Least privilege.** github-plan is read-only and cannot read the data buckets. github-deploy can push the image and upload the site, but cannot change infrastructure, touch IAM or billing, read the records, or overwrite data/v1/. Terraform is applied by hand [INF].
- **Private storage.** All four buckets use S3 Block Public Access. CloudFront with origin access control is the only reader of the site bucket [STG].
- **Container.** Non-root user (uid 1000). Egress is HTTPS only, which is why DuckDB's httpfs extension is installed at build time [DOCK].
- **Tamper evidence.** Locked records sit under Object Lock in governance mode. Overriding it needs a specific permission and header, and shows in CloudTrail [STG].
- **Supply chain.** CI fails on any high-severity advisory in what the site ships (npm audit --omit=dev --audit-level=high). Actions are pinned by commit SHA. Dependabot proposes updates weekly. One accepted dev-only advisory (braces) is documented [CI, DEP].
- **Secrets.** The only repository secret is AWS_ACCOUNT_ID, kept so the account number stays out of public logs. It grants nothing. ANTHROPIC_API_KEY is never set [INF, RES].
- **Public repository.** The repository is public. Public text and code carry no credentials.

12.2 Privacy

The data is public football statistics (player names, teams, injury report status, game results) from nflverse. The documents read describe no user accounts, forms, or collection of visitor data, and the site is a static export with no server. No separate privacy document exists in the repository documents read, and none was needed to describe the system as built. If analytics or accounts are added, or the mobile app collects data, this section needs a threat and data inventory.

12.3 Observability

See section 7.11. The standing rule is that every failure announces itself and silence is treated as failure. Logs go to the CloudWatch log group /ecs/nfl-pipeline (streams pipeline/pipeline/<task id>). meta.json carries generated_at and data_as_of for every nflverse dataset, and the site's stale banner reads it [OPS, CON, NSF].

12.4 Licensing and attribution

Data comes from nflverse, licensed CC BY 4.0, and the site credits it. The project is independent and not affiliated with or endorsed by the NFL or any team. It uses no NFL.com data and no player photos or logos. The site states that it provides statistical rankings only, not betting or financial advice [README, SITE]. The licence for the project's own code is not stated in the documents read, and no licence register for dependencies exists in them.

12.5 Cost

What can cost money	Control	Source
Daily Fargate task	One run a day at a fixed schedule. About 90 seconds and a few cents per run.	OPS
S3 storage	Small: raw 140 MB, warehouse 163 MB (about half a cent a month), JSON 1.6 MB, records about 60 KB a week. Lifecycle rules in Terraform; current raw snapshots never expire.	STG, NSF 15
Whole AWS account	Priced at about a dollar a month. $5 budget and billing alarm Bill_Over_5_Dollars in us-east-1, fired on purpose.	README, NSF 1
LLM use	None in the pipeline. The weekly analyst session runs inside the Claude Pro plan. ANTHROPIC_API_KEY is never set.	RES 7.6
If the analyst session moved to AWS	At API rates about $2 to $6 a week as an interactive session, several times the rest of the project's AWS bill. Claude Code on Bedrock researched at about $1 to $3 a week on Sonnet or $2 to $6 on Opus. Not built.	STG
Optional KMS key on the records bucket	About $1 a month plus $0.03 per 10,000 requests (first 20,000 a month free); about $4 a month if used on every bucket.	STG

12.6 Reliability

- **Rebuild, never reuse.** The warehouse is rebuilt from raw on every run. Reusing yesterday's file would make the run depend on it being correct, and a stale or corrupted file produces wrong output that looks fine [STG].
- **One writer.** DuckDB has no concurrent-writer story. The daily task is the only writer of the snapshot; ad hoc tasks only read it [STG].
- **Gates.** A failed test or validation problem writes nothing, so visitors keep the last good data [OPS].
- **Write-once records** with a compare-on-412 rule: identical rewrite is fine because runs are retried; a difference raises [STG].
- **Retries.** Ingest retries transient failures [README]. The kickoff guard makes a retried run safe: an existing lock is published, never recomputed [NSF 18].
- **Fail closed.** A check that cannot run counts as failed (for example a missing stg_schedules) [NSF].

12.7 Time zones

The schedule is in UTC (15:00 UTC, 11:00 AM Eastern in daylight time). nflverse's kickoff_et is a naive Eastern timestamp. The freshness check uses a fixed 5-hour offset and the kickoff guard a fixed UTC-4 all year; both are chosen to err toward lateness or refusal, never toward a false alarm or a late lock. This avoids a timezone database dependency (zoneinfo needs tzdata on Windows) for at most an hour of precision against a 48-hour threshold [OPS, NSF 6, 18].

12.8 Accessibility

Semantic tables with sort state, real links for navigation, a skip link, focus styles, chart text alternatives, and movement shown with arrows and words, never color alone. axe checks for WCAG 2.2 A and AA run on every page in both themes, with a keyboard-only journey and a phone-layout spec. These do not replace testing with a screen reader [WEB].

12.9 Dependencies

Held-back dependencies and the condition for removing each are in appendix F. Nothing is ignored without a written reason [DEP].

13 Testing strategy

Level	What	Where
Unit and integration (Python)	268 tests. Fakes in pipeline/tests/conftest.py; no network. dbt integration tests included.	CI job test; local pytest
Storage contract	One contract test run against both the local and S3 fakes: a second write is refused, an identical rewrite reports already-present, listing returns what was written. A fake that quietly overwrites would make write-once worthless.	pipeline/tests [STG]
Data tests (dbt)	158 checks on every run, including tests that prove a feature never sees its own week. A drop in the count is a defect.	dbt build; CI and every daily run
No lookahead	A ranking test; feature tests that change a future week's result and check earlier features do not move.	pytest, dbt tests
Freshness gate	9 tests, including the three cases that must not block a publish (a game in progress on a Sunday run, preseason and playoff games, the offseason). The 6 that assert a failure were each confirmed to fail with the check disabled.	pytest [NSF]
Kickoff guard	25 tests. The 5 that assert a refusal were each seen to fail with the guard disabled. 5 more cover the no-roster case; 4 were seen to fail with the check disabled.	pytest [NSF 18, 19]
Web unit	59 tests: formatting, sorting and filtering, team colors and their contrast, chart helpers, the data loader (with a regression test for a shared-request bug).	CI job web [WEB]
Web end-to-end	91 tests against fixture data in web/e2e/fixtures (refresh with node scripts/make-fixtures.mjs). The stale banner appears at 37 hours, is absent at 35, and follows generated_at, not data_as_of.	Playwright; CI job web
Accessibility and phone layout	axe WCAG 2.2 A and AA on every page in both themes; keyboard-only journey; no sideways scrolling and large enough touch targets.	Playwright specs a11y and mobile
Infrastructure	terraform fmt -check and validate for infra/ and infra/bootstrap/ in CI (no AWS needed). Terraform plan on pull requests that touch infra/.	CI jobs infra, plan.yml
Alert paths	Every alarm set to ALARM by hand and its email seen. A task run with an invalid command exited 2 and the failure email arrived.	AWS, 2026-10-05 [NSF]
Type and lint toolchain	A planted type error was seen to fail the TypeScript 7 typecheck. Planted lint errors (conditional hook, missing key, unused variable) were still reported after the ESLint 10 wrapper.	[DEP]
Model evaluation	Walk-forward across 2021 to 2025; held-out weeks for every candidate; margin over baselines and interval coverage.	research/, nfl-pipeline experiment
Dependency security	npm audit --omit=dev --audit-level=high fails the build on a high-severity advisory in what the site ships.	CI job web

Known untested paths: the aws dbt target had not been tested against real S3 when the storage design was first built (2026-10-05); the real warehouse-snapshot fallback token path has not been exercised (OQ-8).

14 Alternatives considered

ID	Decision	Options weighed	Source
D-01	Warehouse handling	Always rebuild, then publish a read-only snapshot (chosen) vs reuse yesterday's file	STG 1
D-02	Reading raw	dbt reads S3 directly through httpfs (chosen) vs copying raw to the task disk	STG 2
D-03	Write-once enforcement	Conditional write plus versioning and Object Lock (chosen) vs a path.exists check, or either control alone	STG 3
D-04	Object Lock mode	Governance (chosen) vs compliance, which cannot be overridden even by the account root	STG 3
D-05	Public copy of records	S3 as runtime store with a scheduled git mirror (chosen; mirror not built) vs git only	STG 4
D-06	Storage API	Add put_bytes_if_absent and list_keys to Storage (chosen) vs a second abstraction	STG 5
D-07	Buckets	One per lifecycle: raw, records, site, tfstate (chosen) vs one shared bucket	STG
D-08	Publishing path	data/v1/ in the site bucket, one origin (chosen) vs a separate published/v1 and a second CloudFront origin	STG
D-09	Encryption	SSE-S3 (documented) vs a KMS key on records only (about $1 a month) vs KMS everywhere (about $4)	STG
D-10	Backtest and experiment	On-demand Fargate tasks on the same image (chosen) vs terminal commands on the laptop	STG
D-11	Analyst session	Laptop in the Pro plan (current) vs automate with API keys vs Claude Code on Bedrock (researched; subscription login from AWS not recommended)	STG
D-12	Infrastructure as code	Terraform (project override) vs CDK or CloudFormation	CLAUDE
D-13	Applying Terraform	By hand from the laptop, CI plans only (chosen) vs CI applies	INF
D-14	Model structure	Direct ridge plus LightGBM ensemble (chosen) vs a layered model (team environment, volume shares, efficiency, touchdowns). Not tested whether the layered model would have helped.	RES 7.3
D-15	Training and publishing populations	Train wide, calibrate on the published population, publish narrow (chosen) vs train narrow vs everyone	RES 5.15, 5.16
D-16	Getting better over time	A governed experiment loop (chosen) vs adaptive ensemble weights, bias correction, per-player bias learning and role-change heuristics, all of which made predictions worse	RES 5.11, 5.12
D-17	Where an LLM fits	Feature proposals through an automated gate (chosen) vs an LLM producing or adjusting numbers	RES 7.6
D-18	Ranking efficiency weight	20% (chosen on 2021 to 2023, confirmed on 2024 and 2025) vs 0% to 100% alternatives	BT
D-19	Site rendering	Static export with data fetched in the browser (chosen) vs rebuilding the site on every data refresh	WEB
D-20	Charts and UI	Hand-written SVG and a small component set (chosen) vs a chart or UI library	WEB
D-21	Freshness gate precision	Fixed Eastern offset, errors toward lateness (chosen) vs a timezone database	NSF
D-22	Missing-data handling in alarms	breaching for staleness alarms; notBreaching for the two snapshot alarms, where no event is healthy	NSF
D-23	TypeScript version	Split: 6.0.x for tools, 7.0.x native compiler for typecheck	DEP
D-24	Query layer	Glue and Athena deferred past launch; they will read the raw bucket	STG
D-25	Lock timing	Lock only inside 24 hours to 30 minutes before first kickoff, refuse otherwise (chosen) vs lock on any run	OPS, NSF 18
D-26	Data source	nflverse release files only (chosen) vs NFL.com	CLAUDE

15 Risks

Risk	Handling	Source
Stale data publishes as if current (for example a wrongly skipped ingest)	Freshness gate checks A and B inside the validation gate; no-success alarm at 26 hours; site banner at 36 hours	NSF 6, 10, 14
A forecast is locked after kickoff	Kickoff guard refuses and exits 1; 25 tests	NSF 18
Nothing can be projected for a new week (roster not published)	Gap noted while kickoff is over a day away; run fails loudly once the lock window opens; options if it arrives too late: fall back to the newest roster week, or a second daily run	OPS, NSF 19; OQ-1
The pipeline dies completely and stops emitting	Staleness alarm with treat-missing-data breaching	NSF 2
The task never starts	ECS and EventBridge failure alarms	NSF 8
An alert path is configured but silent	Every alarm fired by hand once; confirmed SNS subscriptions; ActionsEnabled asserted	NSF 3 to 5
A locked forecast is changed or deleted	Conditional write, versioning, Object Lock governance 365 days, CloudTrail	STG
The git mirror of records stops growing, or does not exist	S3 with Object Lock is authoritative, so nothing is lost. The mirror job is not built (OQ-2)	NSF 12
A corrupted or stale warehouse produces plausible wrong output	Rebuilt from raw on every run and never read back; one writer	STG
A lifecycle rule expires raw snapshots still needed	Rules in Terraform; current snapshots never expire; only noncurrent versions after 30 days	NSF 15
A schema change breaks the site	SCHEMA_VERSION bump; validation against the contract; site refuses unknown versions	NSF 17
The model loses to a plain average	Population restriction; report card shows losing weeks; margin reported with player count	RES 5.15, 7.8
Early-season accuracy is weakest	Stated on the site and in limits; skill measured by period of season	RES 5.11, 7.9
Weather features use typical values for future games	Documented weakness; forecast source not yet connected (OQ-5)	RES 7.3
The analyst session depends on the laptop	Predictions still publish if a session never happens; moving it to AWS is researched (OQ-4)	RES 7.6, STG
Terraform applied by hand can drift or err	CI plan on every infra change, read-only plan role, fmt and validate in CI	INF
Dependency advisories and held-back versions	CI audit gate on shipped dependencies; dependency table with removal conditions	DEP
nflverse changes format or availability	Ingest checks non-empty Parquet and the validation gate refuses bad data; no weekly source smoke test is described in the documents read	README, NSF
A late injury or coaching decision invalidates a locked forecast	Late scratches are not graded against the points model; accepted limit	RES 7.2, 7.9

16 Open questions and known gaps

These come from the repository documents' own lists of gaps and deferred work. Nothing here is invented; each cites where it is stated.

ID	Question or gap	Type	What answering it needs	Source
OQ-1	How soon after a week ends does the next week's roster appear? The one refresh seen was at 11:58 AM Eastern on 2026-10-05, after the 11:00 AM run. If week 5's roster arrives too late for a Thursday game, the options are a fallback to the newest roster week or a second daily run.	Spike	Watch week 5 and later weeks	OPS
OQ-2	The job that copies locked forecasts and the ledger from S3 into git is not built. The public git record does not update on its own, and the ledger in git is only a seed.	Build	A scheduled GitHub Actions job plus a check comparing the newest locked week in S3 and git	README, NSF 12
OQ-3	No frozen comparison model is fitted, so the report card cannot yet show whether the model improves over time. No week had been graded when the README was last updated.	Build	A model frozen at the start of a season; a first locked week finishing	README, RES 7.8
OQ-4	Move the weekly analyst session to AWS? Claude Code on Bedrock, authenticated by the Fargate task role, writing a proposed change as a pull request.	Decision	John's choice, with the cost check ($1 to $3 or $2 to $6 a week at API rates)	STG
OQ-5	A forecast weather source and a stadium table are not built.	Build	A forecast data source	RES 7.3, 8
OQ-6	Component predictions, the layered model and the teammate-target effect are measured or planned and not built.	Research	Held-out tests; each factor must earn its place	RES 8
OQ-7	Defensive player rankings and the mobile app (phase 2) are planned.	Build	The published JSON contract is the base for both	SITE, README
OQ-8	The real log-token path for a warehouse-snapshot fallback has not been exercised; only the alarms were fired by hand.	Test	A real fallback	NSF 13
OQ-9	Whether the records bucket gets a customer-managed KMS key (about $1 a month).	Decision	John's choice	STG
OQ-10	Glue and Athena are deferred past launch and will read the raw bucket.	Deferred	A reason to query raw directly	STG
OQ-11	Phase 1E (launch polish) has no stated scope or status in the documents read.	Gap	The build plan, which is outside the repository	CLAUDE

17 Glossary

Term	Meaning
nflverse	The open NFL data project whose Parquet release files are this project's only source
PPR	Point per reception fantasy scoring. Kickers get common standard scoring because nflverse does not score them.
Composite score	A 0 to 100 blend of efficiency (20%) and production (80%), each metric percentile-scored within the position
Efficiency and production	Rates per opportunity (such as EPA per dropback) versus totals and shares (such as yards and target share)
EPA	Expected points added per play
Spearman	Rank correlation; 0 means no relationship, 1 is perfect
MAE	Mean absolute error, in fantasy points
Margin	Baseline MAE minus model MAE on the same rows; positive means the model is better
Baseline	A simple comparison model: recent average, season average, last ten games, or points per game
Walk-forward	Evaluation that replays time in order, so no prediction sees later data
Conformal prediction	A correction measured on unseen weeks that makes an 80% range cover about 80% of outcomes
Interval coverage	The share of outcomes that land inside the stated range
Lock	Writing a week's projections once, before the first kickoff, so they cannot change
Report card	The public page grading locked forecasts against what happened and listing every experiment
Ledger	The append-only record of every experiment, including failures
Promotion gate	The held-out test a candidate feature must pass before it can ship
Derived state / record	Derived: reproducible and disposable. Record: its value is that it cannot change after the fact.
Object Lock (governance mode)	An S3 setting that blocks deletion or overwrite of an object version for a set time unless a specific permission and header are used
Conditional write	An S3 write with If-None-Match: * that succeeds only if the key does not exist (412 otherwise)
OIDC	OpenID Connect; lets GitHub Actions get short-lived AWS access without stored keys
Fargate	AWS service that runs containers without managing servers; the daily task runs here
EventBridge	AWS scheduler that starts the daily task
CloudFront and OAC	AWS CDN, and origin access control, which makes it the only reader of the private site bucket
dbt	Tool that builds layered data models with tests
DuckDB	The embedded analytical database the warehouse uses
Parquet	Columnar file format used by nflverse and the raw snapshots
MinIO	A local S3 stand-in used to try the pipeline against object storage
Contract	The pydantic models in contract.py that define the published JSON; SCHEMA_VERSION marks breaking changes
Stale banner	A site banner shown when published data is older than 36 hours
Kickoff guard	The check that refuses to lock a week outside 24 hours to 30 minutes before first kickoff
C4	A model for architecture diagrams: context, containers, components
Candidate feature	One proposed new model input, written by Claude Code and judged by the gate

Appendix A: Ranking metrics by position

From dbt/seeds/ranking_weights.csv. Every metric has weight 1 within its component. Direction is higher unless marked lower [SEED].

Position	Efficiency metrics	Production metrics
QB	epa_per_dropback, cpoe, yards_per_attempt, sack_rate (lower), int_rate (lower)	passing_epa, passing_yards, passing_tds, rushing_yards
RB	rushing_epa_per_carry, yards_per_carry, receiving_epa_per_target, rush_yards_over_expected_per_att	rushing_yards, carries, targets, total_tds, first_downs
WR	receiving_epa_per_target, racr, yards_per_target, catch_rate, avg_separation, yac_over_expected	receiving_yards, receptions, receiving_tds, target_share_avg, air_yards_share_avg, wopr_avg
TE	receiving_epa_per_target, racr, yards_per_target, catch_rate, avg_separation, yac_over_expected	receiving_yards, receptions, receiving_tds, target_share_avg, air_yards_share_avg, wopr_avg
K	fg_pct, fg_pct_40_plus, pat_pct	fg_made, fg_made_50_plus, kicker_points

Appendix B: Backtest tables

Generated 2026-10-06T05:24 UTC. Tuning seasons 2021 to 2023, held-out 2024 and 2025. Lift is extra next-week points scored by the top-ranked players (top 12 QB, TE and K; top 24 RB and WR) versus the average ranked player [BT].

Next-week fantasy points (the outcome used to choose the weight), average across positions

Method	Tuning Spearman	Tuning lift	Held-out Spearman	Held-out lift
Composite, 0% efficiency	0.294	2.12	0.302	2.11
Composite, 20% efficiency (chosen and current)	0.295	2.11	0.304	2.14
Composite, 40% efficiency	0.282	1.97	0.290	2.02
Composite, 50% efficiency	0.267	1.94	0.276	1.93
Composite, 60% efficiency	0.246	1.77	0.254	1.68
Composite, 70% efficiency	0.219	1.59	0.227	1.54
Composite, 80% efficiency	0.191	1.38	0.197	1.38
Composite, 100% efficiency	0.136	0.97	0.136	1.02
Baseline: season fantasy points	0.283	2.07	0.300	2.05
Baseline: fantasy points per game	0.294	2.06	0.308	2.12

Next-week EPA (an efficiency-flavored outcome; kickers excluded)

Method	Tuning Spearman	Held-out Spearman
Composite, 0% efficiency	0.116	0.129
Composite, 20% efficiency (chosen and current)	0.124	0.138
Composite, 40% efficiency	0.129	0.143
Composite, 50% efficiency	0.130	0.141
Composite, 60% efficiency	0.128	0.136
Composite, 70% efficiency	0.123	0.133
Composite, 80% efficiency	0.117	0.128
Composite, 100% efficiency	0.105	0.107
Baseline: season fantasy points	0.111	0.125
Baseline: fantasy points per game	0.109	0.124

Held-out results by position (fantasy points, Spearman, standard error over weeks)

Position	Weeks	Chosen and current	Per-game baseline	Composite minus baseline (paired by week)
QB	34	0.244 +/- 0.032	0.253 +/- 0.038	-0.009 +/- 0.019
RB	34	0.475 +/- 0.020	0.500 +/- 0.017	-0.025 +/- 0.009
WR	34	0.395 +/- 0.021	0.379 +/- 0.021	+0.016 +/- 0.009
TE	34	0.332 +/- 0.025	0.308 +/- 0.030	+0.023 +/- 0.012
K	34	0.076 +/- 0.041	0.102 +/- 0.044	-0.026 +/- 0.021

Weeks in one season are not independent, so the standard error is a rough guide [BT].

Held-out Spearman by point in the season (fantasy points, average across positions)

Weeks	Composite, 20% efficiency	Baseline: points per game	Baseline: season points
1 to 5	0.248	0.239	0.227
6 to 11	0.299	0.311	0.301
12 and later	0.357	0.363	0.359

Appendix C: Prediction research summary

Twelve findings from [RES section 1], condensed. All numbers were computed on regular seasons 2021 to 2025 by the scripts in research/.

1 A player's recent usage does most of the work. Adding betting lines, opponent, weather and injury tags moved explained variance of next-game fantasy points from 0.27 to only 0.29.

2 Volume is predictable, yards somewhat, touchdowns barely. Held-out R-squared: carries 0.44 (RB), targets 0.31 (WR), attempts 0.23 (QB); yards 0.12 to 0.29; touchdowns 0.00 to 0.09. Interceptions cannot be predicted.

3 Home-field advantage is real but small (about 0.5 points, 4.8%; quarterbacks about 1.3, 8.1%). No stable loud-stadium effect was found: correlations between 2021-23 and 2024-25 were 0.11, -0.25 and -0.04, none significant.

4 Wind and cold matter for the game, not for the individual player. Adding weather to the per-player model made it slightly worse.

5 The betting line is well calibrated (slope 1.02, r = 0.40).

6 A simple model is nearly as good as a complex one: ridge R-squared 0.280, gradient boosting 0.288, their average 0.289, a plain recent average 0.232.

7 Injury reports are informative: Out played 0.1% of the time, Doubtful 0.4%, Questionable 63% (quarterbacks 35%).

8 Ranges need calibrating: raw quantile models covered 77% for an 80% interval; the conformal correction brought it to 79.7%. The typical 80% range is about 17 fantasy points wide.

9 There is little headroom: an oracle that knew each player's true season-long average scores MAE 5.074 against the model's 5.445, a gap of 6.8%. Error is spread evenly across situations.

10 The model does not get smarter week by week in the usual sense. Weekly retraining beat a frozen model by only 0.037 MAE (0.7%) and the gap did not grow. Adaptive weights, per-position bias correction, per-player bias learning and role-change heuristics all made predictions worse.

11 Who is predicted matters as much as how. The first full run beat the baseline by 0.052, short of the 0.15 bar. Restricting to players averaging at least 4 points clears it.

12 Where the restriction is applied matters. Train wide, calibrate on the published population, publish narrow: margin +0.170, coverage 0.797.

Model families (held-out 2024 and 2025)

Model	MAE	R-squared
Recent-average baseline	5.670	0.232
Ridge regression	5.480	0.280
LightGBM	5.440	0.288
Average of ridge and LightGBM	5.441	0.289

Does the system get smarter week by week? (36 replayed weeks, 7,445 player-games)

Strategy	MAE	Versus frozen
Recent-average baseline	5.670	-0.187
Frozen (trained once on 2021 to 2023)	5.483	baseline
Retrained every week	5.446	+0.037
Retrain plus per-position bias correction	5.448	+0.035
Retrain plus adaptive ensemble weights	5.476	+0.007

Questionable players: did they play?

Report status	Played
Out	0.1%
Doubtful	0.4%
Questionable	63% (QB 35%, K 63%, RB 64%, TE 66%, WR 67%)
Questionable, full practice	71%
Questionable, limited practice	65%
Questionable, did not practice	43%

Reading injury text did not help availability calls (AUC 0.672 with practice status and history, 0.663 after adding injury type). The text nflverse publishes is a body part, not the richer reporting that would help [RES 5.14].

Appendix D: Operations runbook

How the deployed system runs, and what to do when something goes wrong [OPS]. Sign in with aws sso login --profile nfl_player_stats, then set $env:AWS_PROFILE = "nfl_player_stats" (about 12 hours).

When an email arrives

Email	What it means	What to do
NFL pipeline task FAILED ... stopCode=EssentialContainerExited	The task ran and exited non-zero	Read the log. The last lines say which step failed.
NFL pipeline task FAILED ... TaskFailedToStart	The container never started: image pull, role or networking	Check the ECR image exists and the task definition's roles in infra/ecs.tf
Alarm nfl-pipeline-no-success-in-26h	No run has finished with exit code 0 for 26 hours	Find out why runs fail or stop arriving. Check the schedule rule is ENABLED.
Alarm nfl-pipeline-rule-delivery-failed-*	EventBridge could not start the task or deliver an event	Check the rule's target and the scheduler role in infra/monitoring.tf
Alarms nfl-pipeline-fallback, nfl-pipeline-upload_failed	The warehouse snapshot fell back to a rebuild, or its upload failed	Look for WAREHOUSE_SNAPSHOT_FALLBACK or WAREHOUSE_SNAPSHOT_UPLOAD_FAILED in the log

Messages in the log

Message	Meaning
run stopped: dbt build failed, so nothing was published	A data test failed. The old data stays live.
validation failed with N problem(s); nothing was published	The publish gate refused the data, including stale data. The old data stays live.
predict failed: 2026 week N was never locked and its first kickoff ... has passed	The week missed its window. Nothing was locked and that week will not be graded. It repeats each day until the week ends.
predict failed: ... week N is scheduled but no players could be projected	The roster for that week is not published yet. See OQ-1.
predict refused: ... already locked with different projections	A locked week was about to be changed. This needs a person. Do not overwrite it.

Read a log

```
$task = "<task id from the email>"
aws logs get-log-events --log-group-name /ecs/nfl-pipeline \
    --log-stream-name "pipeline/pipeline/$task" --query "events[].message" --output text
```

Run something by hand

```
cd infra
.\run-task.ps1 daily        # what the schedule runs
.\run-task.ps1 run          # ingest, build and publish, no projections
.\run-task.ps1 predict      # project the coming week without locking it
.\run-task.ps1 backtest     # score the rankings; writes backtest/summary.json to the raw bucket
.\run-task.ps1 experiment   # write the failure report for the last scored week
```

To lock a week yourself, run .\run-task.ps1 predict --lock **before that week's first kickoff**. A locked week cannot be rewritten.

Changing things

- **Code or site:** open a pull request. main is protected, so test, web and infra must pass. Merging deploys.
- **Infrastructure:** edit infra/, open a pull request (CI posts the plan), then apply from the laptop with terraform apply. CI never applies.
- **After changing the image:** the next scheduled run uses it. To publish sooner, run daily by hand.
- **A page looks stale after a deploy:** clear CloudFront with aws cloudfront create-invalidation --distribution-id E1OZ5WDKRFMJ25 --paths "/*".

Appendix E: Command reference

Command	What it does
nfl-pipeline run	Ingest, dbt build with tests, validate, publish. Stops at the first failing step.
nfl-pipeline daily	run, then predict --lock-when-due. This is what AWS runs.
nfl-pipeline predict	Project the coming week, grade finished ones, publish both. --lock only before the first kickoff. --lock-when-due is the unattended form.
nfl-pipeline experiment	Write docs/last-week.md, or run one candidate through the promotion gate with --run NAME.
nfl-pipeline backtest	Score the rankings against the following week. Needs a built warehouse. Writes docs/backtest.md.
nfl-pipeline ingest, publish	Run those two steps alone.
dbt build --project-dir dbt --profiles-dir dbt	Run dbt from the repository root with the venv Scripts on PATH.
docker compose run --rm --build pipeline-local run	Run the pipeline in Docker with no Python installed.
docker compose up -d minio minio-init	Start the local S3 stand-in. Stop with docker compose down (add -v to delete stored data).
pytest, ruff check ., ruff format .	Python tests, lint, format. 268 tests.
cd web, npm run dev, lint, typecheck, test, test:e2e	Website development and checks. npm run research regenerates the research page.

Appendix F: Held-back dependencies

Item	State and reason	Remove when
TypeScript 7	Split: 6.0.x for the tools, 7.0.x native compiler for typecheck (installed as the typescript-native alias, about 6 seconds). The TS 7 npm package has no programmatic API yet, and typescript-eslint supports only versions below 6.1.0.	typescript-eslint supports TS 7 (typescript-eslint#10940)
@types/node 26	Held at 24. Types should match the Node the site builds with (22 in CI, 24 locally).	The build moves to a newer Node
ESLint 10	Applied with a wrapper (fixupConfigRules from @eslint/compat) because eslint-plugin-react 7.37.5 calls context.getFilename(), which ESLint 10 removed.	eslint-config-next ships a plugin that supports ESLint 10
braces advisory GHSA-vfj7-8cjw-p6xm	Accepted; no fix exists and GitHub auto-dismissed it as dev-only. A crash from deeply nested brace patterns, reached only through the lint toolchain. npm audit --omit=dev reports 0 vulnerabilities. Do not run npm audit fix --force.	braces publishes a patched release

Appendix G: Source discrepancies and coverage gaps

Where sources disagree (the newer or more specific source was used)

- GitHub's rendered repository page, as fetched, showed an older README (status in development, 116 tests, nothing deployed, no website). The raw README on main is current (live on AWS, 158 dbt checks, predictions, report card). This document follows the raw file.
- docs/prediction-research.md section 7.7 says weekly locking is manual, scheduling is part of an AWS deployment not yet done, and no week has been locked. [OPS] and [README] supersede it: the daily schedule has run on AWS since 2026-10-05.
- The research paper reports four ledger entries. The live ledger is in the records bucket and may have grown; the ledger in git is only a seed [NSF 12].
- The accuracy bar in the research paper (coverage 78% to 82% on the final model) and the promotion gate (0.75 to 0.85) are two different checks, both recorded in QR-08.

What was not read

- BUILD_PLAN.md (sections 13 and 19 are cited second-hand through the storage design) and SESSION_CONTEXT.md, which live outside the repository.
- The Terraform files, the pipeline and dbt source, the tests, research/ scripts and results, docs/last-week.md, .claude/, and the workflow files plan.yml and deploy.yml. They are described here only through the documents that cover them.
- Any change after Oct 6, 2026, the date of the newest document read. In particular, whether week 5 or later has been locked and graded, and the current ledger entries.
- The hiking project's own documents were used only as the template for this document's structure.

Appendix H: Sources

Source	Location
Website (project page, home, projects, about)	https://www.johnwasikye.com/projects/nfl-player-performance
Live site (home, methodology, report card)	https://nflstats.johnwasikye.com
Repository	https://github.com/John-Wasikye/nfl-player-performance
Raw files read	README.md, CLAUDE.md, pyproject.toml, Dockerfile, .github/workflows/ci.yml, pipeline/nfl_pipeline/contract.py, dbt/seeds/ranking_config.csv, dbt/seeds/ranking_weights.csv, docs/storage-design.md, docs/no-silent-failures.md, docs/prediction-research.md, docs/backtest.md, docs/operations.md, docs/dependencies.md, infra/README.md, web/README.md
Template	https://github.com/John-Wasikye/will-i-have-signal-out-there (docs/platform/design-doc.md and companions, branch feature/platform-skeleton)
Data	https://github.com/nflverse/nflverse-data (CC BY 4.0)